

claude-windows / 744270aa-6a38-4011-b018-cdd9aae94849

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\744270aa-6a38-4011-b018-cdd9aae94849\subagents\agent-a67a8bee8aa546faf.jsonl`
- SHA-256: `947cef21ca3897cddeda9899bd648197adf976fba836e909c558aaf4ba6c2d254`
- Source modified: `2026-07-09T02:46:59+00:00`
- Imported at: `2026-07-09T08:31:56+00:00`
- project: `subagents`
- session_id: `744270aa-6a38-4011-b018-cdd9aae94849`

Transcript

1. user (2026-07-09T02:38:50.001Z)

You are exploring the repo at C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer (a Python badminton highlight indexer). I need a precise map of how to run the FULL end-to-end pipeline LOCALLY from the CLI with NO web/SPA and NO user interaction, and what timing/performance instrumentation already exists. Do NOT modify any files ? read only.

Answer these specific questions with exact file:line citations and copy the relevant code/CLI snippets:

1. ****End-to-end CLI flow.**** Entry point is `python -m backend.main`. Read `backend/main.py` (argument parser + main flow) and `backend/orchestration.py`. Map the pipeline stages in order: (a) checksum/MD5 hashing of the input video, (b) validation (validators), (c) segmentation/indexing (segmenters), (d) stitching/compiling the reel. For each stage, cite the function(s) that run it and how they're timed/logged if at all.
2. ****Stitching the FULL reel from CLI with no user step.**** The user wants to always stitch the entire reel (all rallies) automatically, no SPA click. Find how a reel is compiled: look at `backend/pipeline/stitcher/compiler.py` and any `/api/compile` handler. Is there a CLI flag on `backend.main` to compile a reel after processing (e.g. `--stitch`, `--compile`, `--make-reel`)? If not, what is the exact programmatic call (function + args) to compile a full reel of all indexed rallies for a given video_id? Cite it.
3. ****Existing performance/latency instrumentation.**** Search for any existing timing, benchmarking, profiling, or "observability report" code: grep for `time.perf\_counter`, `time.monotonic`, `elapsed`, `duration\_ms`, `latency`, `timing`, `benchmark`, `perf`, `stopwatch`, `observability`, `PhaseTimer`, `Timer`. Report what stage-level timings the pipeline already emits (e.g. does orchestration already record per-phase durations to the DB or a report?). Cite files/lines.
4. ****Segmenters and local-GPU feasibility.**** List the segmenters in `backend/pipeline/segmenters/`. Which run fully locally with no cloud/API key (motion, yolo_hybrid, wasb_hybrid, tracknet_hybrid) vs which need cloud (gemini)? Which use the GPU (torch/CUDA)? The local machine has an 8GB RTX 2070 GPU. Note the `default\_segmenter` in config.json and any per-run `--segmenter` flag.
5. ****Config surface for a deterministic repeatable run.**** In config.json and backend/config, what knobs control: segmenter selection, validation thresholds (blur/court/relevance), stitching (padding, min_shot_count, watermark), and compute placement (compute_target=local vs cloud_run)? I want to force everything LOCAL and deterministic. Cite the config keys.
6. ****Existing regression/nightly harness.**** Look at `run\_eval.py`, `run\_regression.py`, `run\_all\_tests.py`, and anything under `output/nightly\_regression/` or `scripts/` or `tools/` that looks like a repeatable eval/regression/benchmark runner. Summarize what each does (accuracy vs performance) and whether any already produce per-run timing numbers.

Return a structured report with headings for each of the 6 questions, exact file:line citations,

and copied code snippets for the key call sites (checksum, validate, segment, stitch, and any existing timer). Be concrete ? I will use this to build a repeatable performance-benchmark script.

2. assistant (2026-07-09T02:38:57.396Z)

I'll explore the repository systematically to answer all 6 questions. Let me start by mapping the structure and reading the key entry-point files.

3. user (2026-07-09T02:39:00.058Z)

```
backend\eval\__init__.py
backend\utils\__init__.py
tests\__init__.py
backend\tools\__init__.py
backend\eval\audio\__init__.py
backend\pipeline\audio\__init__.py
tests\test_hashing.py
backend\features\__init__.py
tests\test_import_smoke.py
training\gen0\__init__.py
training\__init__.py
scratch\run_gx010125.py
scratch\probe_cv2_count.py
scratch\probe_gemini.py
scratch\repro_drawtext_font.py
scratch\repro_movie_overlay.py
scratch\stitch_min5.py
scratch\debug_wsl.py
tools\__init__.py
scratch\trim_video.py
scratch\p3_smoke.py
scratch\ab_eval_demo.py
scratch\ab_n6_lovo.py
scratch\run_gx010125_rescore.py
scratch\run_gx010125_full.py
backend\utils\hashing.py
scratch\quality_report.py
scratch\gate_ab_ablation.py
scratch\gate_ab_detail.py
scratch\served_gateA_verify.py
scratch\served_gateA_sweep.py
backend\pipeline\personalization\__init__.py
scratch\make_product_overview_pdf.py
scratch\make_boxhill_highlights.py
scratch\verify_highlights.py
scratch\compare_local_vs_gemini.py
scratch\at_par.py
scratch\r1_evidence.py
scratch\r5_evidence.py
scratch\gemini_long_detect.py
scratch\analyze_19june.py
scratch\extract_19june.py
scratch\gemini_detect_19june.py
scratch\compare_19june_report.py
scratch\extract_person_19june.py
scratch\analyze_multishuttle_142.py
scratch\reconstruct_prominent_track_142.py
scratch\extract_multishuttle_frames_142.py
scratch\overlay_tracks_142.py
scratch\validate_multishuttle_142.py
scratch\multitrack_select.py
scratch\person_court_142.py
scratch\run_wasb_gx128.py
scratch\eval_multitrack_19june.py
```

```

scratch\keepawake.py
scratch\watchdog_gx128.py
scratch\eval_multitrack_gx128_golden.py
scratch\make_highlight.py
scratch\make_cloud_manifest.py
scratch\migrate_golden_to_gcs.py
scratch\measure_gate_a.py
scratch\measure_served_gate_a_15clip.py
backend\api\__init__.py
backend\pipeline\segmenters\__init__.py
backend\pipeline\validators\__init__.py
backend\results.py
backend\sports\base.py
backend\utils\redact.py
backend\worker\__init__.py
tests\test_reliability_hardening.py
artifacts\cloudrun_e2e_validation\_scratch\make_reel.py
artifacts\landing\make_og.py
artifacts\so.py
backend\eval\gemini_refine.py
backend\annotations\automated_provider.py
backend\annotations\base.py
backend\annotations\file_provider.py
backend\annotations\__init__.py
backend\api\auth.py
backend\billing.py
backend\config\loader.py
backend\eval\annotations.py
backend\eval\audio\e1_probe.py
backend\eval\batch.py
backend\eval\calibrate_wasb.py
backend\eval\calibration.py
backend\eval\classifier.py
backend\eval\eval_partial_labels.py
backend\eval\eval_stats.py
backend\eval\experiment.py
backend\eval\export_wasb_segments.py
backend\eval\fusion_compare.py
backend\eval\golden_fixtures.py
backend\eval\golden_real_fixtures.py
backend\eval\gt_loader.py
backend\eval\harness.py
backend\eval\metrics.py
backend\eval\per_user_eval.py
backend\eval\per_user_gate.py
backend\eval\per_user_regression.py
(Showing 100 of 374 matching files; 274 more are not listed. Narrow the pattern or path to see
the rest.)

```

4. user (2026-07-09T02:39:00.715Z)

```

total 734
drwxr-xr-x 1 avidu 197609    0 Jul  8 21:45 .
drwxr-xr-x 1 avidu 197609    0 Jul  8 12:07 ..
drwxr-xr-x 1 avidu 197609    0 Jul  8 10:35 .claude
-rw-r--r-- 1 avidu 197609 94208 Jul  8 16:00 .coverage
-rw-r--r-- 1 avidu 197609   611 Jun 11 10:30 .coveragerc
-rw-r--r-- 1 avidu 197609    69 Jun 10 23:25 .env
-rw-r--r-- 1 avidu 197609 1059 Jul  7 18:29 .gcloudignore
drwxr-xr-x 1 avidu 197609    0 Jul  9 08:06 .git
-rw-r--r-- 1 avidu 197609   947 Jun 25 16:24 .gitattributes
drwxr-xr-x 1 avidu 197609    0 Jul  2 08:13 .github
-rw-r--r-- 1 avidu 197609 1998 Jul  2 08:13 .gitignore
drwxr-xr-x 1 avidu 197609    0 Jun 11 10:42 .mypy_cache
drwxr-xr-x 1 avidu 197609    0 May 19 15:58 .pytest_cache

```

```

drwxr-xr-x 1 avidu 197609      0 Jun 11 10:32 .ruff_cache
drwxr-xr-x 1 avidu 197609      0 Jun 25 16:24 .vscode
drwxr-xr-x 1 avidu 197609      0 Jul  7 10:27 __pycache__
drwxr-xr-x 1 avidu 197609      0 Jul  4 00:22 artifacts
drwxr-xr-x 1 avidu 197609      0 Jul  8 14:15 backend
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 backend.db
-rw-r--r-- 1 avidu 197609 6619 Jul  2 11:50 CLAUDE.md
-rw-r--r-- 1 avidu 197609 3034 Jul  8 13:48 config.json
drwxr-xr-x 1 avidu 197609      0 Jun 25 16:24 deploy
drwxr-xr-x 1 avidu 197609      0 Jul  7 22:13 docs
drwxr-xr-x 1 avidu 197609      0 Jul  4 06:58 eval_baselines
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 indexer.db
-rw-r--r-- 1 avidu 197609 1999 Jun 25 16:24 mypy.ini
drwxr-xr-x 1 avidu 197609      0 Jul  8 16:00 output
-rw-r--r-- 1 avidu 197609 5413 Jul  4 13:58 pyproject.toml
-rw-r--r-- 1 avidu 197609 240 May 30 14:08 pytest.ini
-rw-r--r-- 1 avidu 197609 77824 Jun 21 15:51 rally_index.db
-rw-r--r-- 1 avidu 197609 36124 Jul  8 13:48 README.md
-rw-r--r-- 1 avidu 197609 1674 Jul  2 20:05 requirements.txt
-rw-r--r-- 1 avidu 197609 585 Jul  2 19:49 requirements-trackers.txt
-rw-r--r-- 1 avidu 197609 809 Jun 25 16:24 ruff.toml
-rwxr-xr-x 1 avidu 197609 2118 Jun 29 13:47 run_all_tests.py
-rw-r--r-- 1 avidu 197609 5739 Jun 29 13:47 run_eval.py
-rw-r--r-- 1 avidu 197609 4594 Jun 29 13:47 run_regression.py
drwxr-xr-x 1 avidu 197609      0 Jul  7 08:50 scratch
drwxr-xr-x 1 avidu 197609      0 Jul  5 19:39 scripts
-rw-r--r-- 1 avidu 197609 2282 Jul  8 10:53 SESSION_HANDOFF (1).md
-rw-r--r-- 1 avidu 197609 1447 Jul  8 18:23 SESSION_HANDOFF.md
drwxr-xr-x 1 avidu 197609      0 Jul  8 14:15 tests
drwxr-xr-x 1 avidu 197609      0 Jul  2 19:37 tools
drwxr-xr-x 1 avidu 197609      0 Jun 10 15:57 training
---BACKEND---
total 276
drwxr-xr-x 1 avidu 197609      0 Jul  8 14:15 .
drwxr-xr-x 1 avidu 197609      0 Jul  8 21:45 ..
drwxr-xr-x 1 avidu 197609      0 Jul  8 15:58 __pycache__
drwxr-xr-x 1 avidu 197609      0 Jun 29 13:47 annotations
drwxr-xr-x 1 avidu 197609      0 Jul  8 12:39 api
drwxr-xr-x 1 avidu 197609      0 Jun 12 15:39 assets
drwxr-xr-x 1 avidu 197609      0 Jun  7 14:14 audio
-rw-r--r-- 1 avidu 197609 3293 Jun 29 13:47 billing.py
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:41 compute
drwxr-xr-x 1 avidu 197609      0 Jul  8 13:48 config
drwxr-xr-x 1 avidu 197609      0 Jun  7 14:14 detectors
drwxr-xr-x 1 avidu 197609      0 Jul  4 06:58 eval
drwxr-xr-x 1 avidu 197609      0 Jun 29 13:47 features
-rw-r--r-- 1 avidu 197609 8102 Jun 29 13:47 flywheel.py
drwxr-xr-x 1 avidu 197609      0 Jul  8 13:48 infrastructure
-rw-r--r-- 1 avidu 197609 76660 Jul  8 13:48 main.py
-rw-r--r-- 1 avidu 197609 80544 Jul  8 14:15 orchestration.py
drwxr-xr-x 1 avidu 197609      0 Jun 14 16:28 pipeline
drwxr-xr-x 1 avidu 197609      0 Jun 29 13:47 providers
drwxr-xr-x 1 avidu 197609      0 Jun 29 13:47 reporting
-rw-r--r-- 1 avidu 197609 3009 Jun 25 16:24 results.py
-rw-r--r-- 1 avidu 197609 5378 Jul  2 15:31 server_runtime.py
drwxr-xr-x 1 avidu 197609      0 Jun 29 13:47 sports
drwxr-xr-x 1 avidu 197609      0 Jun  7 14:14 stitcher
drwxr-xr-x 1 avidu 197609      0 Jul  5 18:38 storage
drwxr-xr-x 1 avidu 197609      0 Jul  2 11:27 tools
drwxr-xr-x 1 avidu 197609      0 Jul  7 22:13 ui
drwxr-xr-x 1 avidu 197609      0 Jul  4 14:45 utils
drwxr-xr-x 1 avidu 197609      0 Jul  8 13:48 worker
---PIPELINE---
total 24
drwxr-xr-x 1 avidu 197609 0 Jun 14 16:28 .

```

```
drwxr-xr-x 1 avidu 197609 0 Jul  8 14:15 ..
drwxr-xr-x 1 avidu 197609 0 Jul  2 13:04 audio
drwxr-xr-x 1 avidu 197609 0 Jul  7 16:57 detectors
drwxr-xr-x 1 avidu 197609 0 Jun 29 13:47 personalization
drwxr-xr-x 1 avidu 197609 0 Jul  7 16:57 segmenters
drwxr-xr-x 1 avidu 197609 0 Jul  8 14:15 stitcher
drwxr-xr-x 1 avidu 197609 0 Jul  7 23:11 validators
```

5. assistant (2026-07-09T02:39:04.771Z)

Let me read the core files and explore the key directories in parallel.

6. user (2026-07-09T02:39:05.955Z)

```
1      """Canonical file-identity hashing for the indexer.
2
3      `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4      (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5      `scripts/run_process_and_stitch.py`)
6      defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7      The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8      any future change to one copy would silently diverge the DB key.
9
10     This module is the single source of truth. Import `compute_video_id` everywhere a
11     video needs to be keyed.
12     """
13
14     from __future__ import annotations
15
16     import hashlib
17
18     # 64 KB matches the value used by the live pipeline (backend/main.py) before this
19     # was centralized. The chunk size does not affect the resulting digest ? it only
20     # bounds memory use while streaming large files.
21     _CHUNK_SIZE = 65536
22
23     def compute_video_id(filepath: str) -> str:
24         """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26         Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27         into memory at once.
28         """
29         hasher = hashlib.md5()
30         with open(filepath, "rb") as f:
31             for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32                 hasher.update(chunk)
33         return hasher.hexdigest()
34
```

7. user (2026-07-09T02:39:08.283Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-flash-latest",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
```

```

13             35,
14             40,
15             40
16         ],
17         "court_green_upper": [
18             85,
19             255,
20             255
21         ],
22         "court_pixels_min_ratio": 0.05
23     }
24 },
25     "indexing": {
26         "default_segmenter": "gemini",
27         "use_gpu": true,
28         "motion_threshold": 0.08,
29         "min_segment_duration": 3.0,
30         "dead_time_merge_gap": 2.0,
31         "chunk_duration": 90,
32         "chunk_overlap": 10,
33         "detector_model": "fasterrcnn_resnet50_fpn",
34         "detector_score_threshold": 0.5,
35         "yolo_velocity_threshold": 0.15,
36         "yolo_frame_skip": 3,
37         "yolo_tracker": "bytetrack.yaml",
38         "yolo_prefetch_workers": 2,
39         "min_action_window_duration": 2.0,
40         "log_yolo_candidates": true,
41         "store_yolo_candidates": true,
42         "ai_handoff_padding": 2.0,
43         "ai_max_workers": 1,
44         "tracknet": {
45             "wsl_distro": "Ubuntu",
46             "repo_dir": "~/models/TrackNetV4",
47             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48             "conda_env": "TrackNetV4",
49             "weights_path": "",
50             "queue_length": 5,
51             "stage_in_wsl": true,
52             "wsl_stage_dir": "~/clips",
53             "timeout_sec": 1800,
54             "velocity_threshold": 0.02
55         },
56         "wasb": {
57             "wsl_distro": "Ubuntu",
58             "repo_dir": "~/models/WASB-SBDT",
59             "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60             "conda_env": "wasb",
61             "weights_path": "~/models/WASB-
62             SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
63             "sport": "badminton",
64             "wsl_stage_dir": "~/clips",
65             "timeout_sec": 1800,
66             "velocity_threshold": 0.02
67         }
68     },
69     "output": {
70         "default_highlights_name": "highlights.mp4",
71         "db_name": "sports_indexer.db"
72     },
73     "stitching": {
74         "begin_padding": 0.5,
75         "end_padding": 0.5,
76         "use_cache": false,
77         "min_shot_count": 1,

```

```

77         "watermark": {
78             "enabled": true,
79             "text": "Generated by Khelsutra.guru",
80             "position": "bottom-right",
81             "opacity": 0.85,
82             "logo_path": "",
83             "font_path": ""
84         },
85         "renditions": {
86             "sd": {
87                 "max_width": 854,
88                 "max_height": 480,
89                 "crf": 30,
90                 "audio_bitrate": "96k",
91                 "video_preset": "superfast",
92                 "pix_fmt": "yuv420p"
93             }
94         }
95     }
96 }
97

```

8. user (2026-07-09T02:39:08.964Z)

```

===segmenters===
total 141
drwxr-xr-x 1 avidu 197609    0 Jul  7 16:57 .
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 ..
-rw-r--r-- 1 avidu 197609   757 Jun 25 16:24 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609   548 Jun 29 13:47 _keyhint.py
-rw-r--r-- 1 avidu 197609  3141 Jun 29 13:47 base.py
-rw-r--r-- 1 avidu 197609 12058 Jul  7 16:57 fusion_hybrid.py
-rw-r--r-- 1 avidu 197609 29193 Jul  6 23:54 gemini.py
-rw-r--r-- 1 avidu 197609   9703 Jun 29 13:47 motion.py
-rw-r--r-- 1 avidu 197609   1836 Jun 29 13:47 tracknet_hybrid.py
-rw-r--r-- 1 avidu 197609  25442 Jul  6 23:54 trajectory_hybrid.py
-rw-r--r-- 1 avidu 197609   2804 Jul  7 16:57 wasb_hybrid.py
-rw-r--r-- 1 avidu 197609  25672 Jul  6 23:54 yolo_hybrid.py
===validators===
total 72
drwxr-xr-x 1 avidu 197609    0 Jul  7 23:11 .
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 ..
-rw-r--r-- 1 avidu 197609   903 Jun 25 16:24 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609  9887 Jul  7 23:11 base.py
-rw-r--r-- 1 avidu 197609   3106 Jul  7 22:13 blur.py
-rw-r--r-- 1 avidu 197609   3629 Jul  2 13:04 format.py
-rw-r--r-- 1 avidu 197609   6097 Jul  7 22:13 frame_sampler.py
-rw-r--r-- 1 avidu 197609   5226 Jul  2 13:04 pts_continuity.py
-rw-r--r-- 1 avidu 197609   4768 Jul  7 22:13 relevance.py
-rw-r--r-- 1 avidu 197609   4065 Jul  2 13:04 resolution_fps.py
-rw-r--r-- 1 avidu 197609   4218 Jul  7 22:13 scene_cut.py
===stitcher===
total 56
drwxr-xr-x 1 avidu 197609    0 Jul  8 14:15 .
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 ..
drwxr-xr-x 1 avidu 197609    0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609 43885 Jul  8 15:41 compiler.py
-rw-r--r-- 1 avidu 197609   3653 Jun 29 13:47 watermark_i18n.py
===config===
total 104
drwxr-xr-x 1 avidu 197609    0 Jul  8 13:48 .
drwxr-xr-x 1 avidu 197609    0 Jul  8 14:15 ..
-rw-r--r-- 1 avidu 197609  1947 Jul  8 11:41 __init__.py

```

```

drwxr-xr-x 1 avidu 197609      0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609 13296 Jun 29 13:47 loader.py
-rw-r--r-- 1 avidu 197609 40521 Jul  8 13:48 models.py
-rw-r--r-- 1 avidu 197609 2911 Jul  8 11:41 placement.py
-rw-r--r-- 1 avidu 197609 7797 Jul  8 11:41 presets.py
-rw-r--r-- 1 avidu 197609 17707 Jul  4 06:58 startup.py
===api===
total 77
drwxr-xr-x 1 avidu 197609      0 Jul  8 12:39 .
drwxr-xr-x 1 avidu 197609      0 Jul  8 14:15 ..
-rw-r--r-- 1 avidu 197609   59 Jun 25 16:24 __init__.py
drwxr-xr-x 1 avidu 197609      0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609 6292 Jun 29 13:47 auth.py
-rw-r--r-- 1 avidu 197609 24951 Jul  8 12:39 job_worker.py
-rw-r--r-- 1 avidu 197609 9139 Jul  5 17:35 jobs.py
drwxr-xr-x 1 avidu 197609      0 Jun 25 16:24 static
-rw-r--r-- 1 avidu 197609 14734 Jul  5 18:38 uploads.py
===compute===
total 56
drwxr-xr-x 1 avidu 197609      0 Jul  8 11:41 .
drwxr-xr-x 1 avidu 197609      0 Jul  8 14:15 ..
-rw-r--r-- 1 avidu 197609 4184 Jul  8 11:41 __init__.py
drwxr-xr-x 1 avidu 197609      0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609 6646 Jul  8 11:41 base.py
-rw-r--r-- 1 avidu 197609 25690 Jul  8 11:41 cloud_run.py

```

9. assistant (2026-07-09T02:39:13.985Z)

Now let me look at the main.py argument parser and main flow, plus the orchestration file structure.

10. user (2026-07-09T02:39:16.013Z)

```

1:import argparse
904:def process_video(req: ProcessRequest, request: Request):
1149:def run_cli_diagnose_clip(video_path: str, timestamp: float):
1271:def main():
1272:    parser = argparse.ArgumentParser(
1275:    parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1276:    parser.add_argument(
1281:    parser.add_argument(
1286:    parser.add_argument(
1291:    parser.add_argument(
1294:    parser.add_argument(
1300:    parser.add_argument(
1303:    parser.add_argument(
1309:    parser.add_argument(
1316:    parser.add_argument(
1321:    parser.add_argument(
1326:    parser.add_argument(
1331:    parser.add_argument(

```

11. user (2026-07-09T02:39:17.536Z)

```

65:def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
78:def _validation_failure_message(failures: List[Any]) -> str:
89:def _report_config_for_input(config: Any, raw_input: str) -> Any:
117:def _ingest_remote_segments(
175:def _local_detector_weights_present(cfg, seg_name: str) -> bool:
199:def _cloud_available(cfg) -> bool:
218:def _will_detect_on_cloud(cfg, req_compute: Optional[str]) -> bool:
236:def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
241:    minutes = int(elapsed_sec) // 60
244:    return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"

```



```

247: def submit_time_video_id(input_ref, cfg) -> Optional[str]:
269: def _looks_like_video_id(value: Optional[str]) -> bool:
283: def _cached_process_payload(
302: def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
323: def _rehydrate_lock(video_id: str) -> threading.Lock:
328: def probe_process_cache(db, cfg, video_id: str) -> bool:
361: def cached_process_result(
439: def _first_object_under(prefix_uri: str, storage_config: dict) -> str:
450: def read_path_rehydrate(db, cfg, video_id: str) -> bool:
464: def _maybe_dispatch_remote(
486:     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
697:         on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
765: def _execute_processing(
1116: def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1141:     def __init__(self, status_code: int, detail: str):
1147: def _resolve_compile(
1196: def reel_object_uri(cfg, video_id: str, name: str) -> str:
1206: def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1214: def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
1227: def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1241: def _compile_wait_message(elapsed_sec: float) -> str:
1244:     minutes = int(elapsed_sec) // 60
1247:     return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1250: def _stitch_reel(
1400: def _dispatch_compile_remote(
1514:     spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1559: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:

```

12. assistant (2026-07-09T02:39:23.735Z)

Let me read the main.py CLI parser and main flow, plus the process_video function.

13. user (2026-07-09T02:39:24.037Z)

```

1271     def main():
1272         parser = argparse.ArgumentParser(
1273             description="Sports Highlight Indexer Orchestrator"
1274         )
1275         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1276         parser.add_argument(
1277             "--output-dir",
1278             default=None,
1279             help="Directory where processed clips/highlights are saved (overrides
config.json output.output_dir)",
1280         )
1281         parser.add_argument(
1282             "--setup",
1283             action="store_true",
1284             help="Launch dependency checker and setup utility",
1285         )
1286         parser.add_argument(
1287             "--debug-clip",
1288             type=float,
1289             help="Timestamp (in seconds) of a video clip to run detailed diagnostics on",
1290         )
1291         parser.add_argument(
1292             "--server", action="store_true", help="Run the FastAPI local API server"
1293         )
1294         parser.add_argument(
1295             "--app",
1296             action="store_true",
1297             help="Launch the app: start the local server (localhost-only) and open the "
1298             "bundled UI in your browser. Falls back to a free port if --port is busy.",
1299         )
1300         parser.add_argument(

```

```

1301         "--port", type=int, default=8000, help="Port to run the FastAPI server on"
1302     )
1303     parser.add_argument(
1304         "--host",
1305         default=None,
1306         help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
1307         "security.edge_auth_enabled. Expose on a network ONLY behind an auth gate.",
1308     )
1309     parser.add_argument(
1310         "--max-frames",
1311         type=int,
1312         help="Limit processing to the first N sub-sampled frames for debugging",
1313     )
1314
1315     available_segmenters = list(segmenter_registry.list_available().keys())
1316     parser.add_argument(
1317         "--segmenter",
1318         choices=available_segmenters,
1319         help=f"Choose rally detection segmenter. Available: {available_segmenters}",
1320     )
1321     parser.add_argument(
1322         "--trim-start",
1323         type=float,
1324         help="Start time in seconds for the debug trim segment feature",
1325     )
1326     parser.add_argument(
1327         "--trim-end",
1328         type=float,
1329         help="End time in seconds for the debug trim segment feature",
1330     )
1331     parser.add_argument(
1332         "--trim-output",
1333         help="Output filename for the debug trimmed segment (saves in output-dir unless
absolute path)",
1334     )
1335
1336     args = parser.parse_args()
1337
1338     if args.setup:
1339         # Invoke the diagnostics/bootstrap script (renamed from the old root
1340         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
1341         # build system). Resolve its path from this file so `indexer --setup`
1342         # works regardless of the caller's cwd.
1343         import subprocess
1344
1345         doctor_path = os.path.join(
1346             os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
1347             "scripts",
1348             "doctor.py",
1349         )
1350         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
1351         subprocess.run([sys.executable, doctor_path])
1352         return
1353
1354     if args.trim_start is not None and args.trim_end is not None:
1355         if not args.video_path:
1356             print("[ERROR] Please provide --video-path to extract a segment.")
1357             return
1358
1359         # Determine output path
1360         out_filename = args.trim_output or "trimmed_segment.mp4"
1361         if os.path.isabs(out_filename):
1362             out_path = out_filename
1363             out_dir = os.path.dirname(out_path)
1364             out_name = os.path.basename(out_path)

```

```

1365         else:
1366             out_dir = resolve_output_dir(
1367                 args.output_dir or "output"
1368             ) # P0a: app-home aware (CWD-identical when unset)
1369             out_path = os.path.join(out_dir, out_filename)
1370             out_name = out_filename
1371
1372             os.makedirs(out_dir, exist_ok=True)
1373             print(
1374                 f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1375             )
1376
1377             # global_config isn't initialized this early; load the typed config so the trim
1378             # clip carries the same configured stitching paddings + watermark as a real
1379             # highlight (watermark default-on).
1380             stitching = load_app_config().stitching
1381             compiler = VideoCompiler(
1382                 out_dir,
1383                 begin_padding=stitching.begin_padding,
1384                 end_padding=stitching.end_padding,
1385                 watermark=stitching.watermark,
1386             )
1387             try:
1388                 res_path = compiler.compile_highlights(
1389                     args.video_path, [(args.trim_start, args.trim_end)], out_name
1390                 )
1391                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1392             except Exception as e:
1393                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
1394             return
1395
1396         if args.debug_clip is not None:
1397             if not args.video_path:
1398                 print("[ERROR] Please provide --video-path to debug a specific clip.")
1399             return
1400             run_cli_diagnose_clip(args.video_path, args.debug_clip)
1401             return
1402
1403         # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly
1404         `--app`
1405         # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1406         # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1407         # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1408         # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1409         # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1410         # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1411         if args.app:
1412             seeded_path, seeded = write_light_default_config()
1413             if seeded:
1414                 print(
1415                     f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
1416                     f"(detector 'motion' ? offline, no API key or GPU needed). "
1417                     f"Use the in-app setup to switch to Gemini."
1418                 )
1419
1420         # Initialize Global Config and DB
global global_config, db_instance

```

```

1421     global_config = load_app_config() # returns typed AppConfig
1422     _enforce_startup_or_exit(
1423         global_config
1424     ) # M5: fail fast on an incoherent ACTIVE profile
1425
1426     # The CLI --output-dir overrides the configured output directory ONLY when
explicitly
1427     # passed. When not passed, the config.json output.output_dir value takes effect.
1428     # It now lives on the typed model (OutputConfig.output_dir), so every consumer ?
including
1429     # /api/compile ? reads the same value instead of a write-only runtime hack.
1430     # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1431     # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1432     if args.output_dir is not None:
1433         global_config.output.output_dir = resolve_output_dir(args.output_dir)
1434     else:
1435         global_config.output.output_dir = resolve_output_dir(
1436             global_config.output.output_dir
1437         )
1438     os.makedirs(global_config.output.output_dir, exist_ok=True)
1439
1440     db_path = os.path.join(
1441         global_config.output.output_dir, global_config.output.db_name
1442     )
1443
1444     # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
1445     # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
1446     # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
1447     # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
1448     global _instance_lock
1449     try:
1450         _instance_lock = acquire_lock(db_path + ".lock")
1451     except OSError as exc:
1452         # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1453         # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1454         # legitimate single instance).
1455         logger.warning(
1456             "[JOBS] Could not create the single-instance lock at %.lock (%s);
proceeding "
1457             "WITHOUT the second-instance guard.",
1458             db_path,
1459             exc,
1460         )
1461         _instance_lock = None
1462     else:
1463         if _instance_lock is None:
1464             print(
1465                 f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1466                 f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
1467                 f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
1468             )
1469             sys.exit(1)
1470
1471     db_instance = Database(db_path)
1472

```

```

1473     # Crash recovery (#16, #329): jobs left queued/running by a previous process are
dead ?
1474     # the executor is in-process. Compile jobs (idempotent CPU stitch) are RE-QUEUED so
a reel
1475     # interrupted by a Cloud Run scale-down finishes on the next drain instead of
failing the
1476     # client; process jobs (GPU, non-idempotent) are failed so pollers see a terminal
state.
1477     requeued, failed = db_instance.recover_stale_jobs()
1478     if requeued or failed:
1479         logger.warning(
1480             f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
1481             f"{failed} process failed."
1482         )
1483     # P3 (#471): md5-keyed RUNNING remote jobs resume on outputs/<md5>/report.json ?
never re-dispatched.
1484     resume_interrupted_remote_jobs(global_config, db_instance)
1485
1486     # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1487     # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1488     # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1489     _maybe_sweep_gemini_orphans(global_config)
1490
1491     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1492     if args.video_path and not args.server and not args.app:
1493         print(f"[CLI] Processing video file: {args.video_path}")
1494         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1495         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1496         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1497         storage_cfg = getattr(global_config, "storage", None)
1498         try:
1499             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1500         except ValueError as e:
1501             print(f"[FAIL] {e}")
1502             return
1503         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1504             print(f"[FAIL] Video file not found: {args.video_path}")
1505             return
1506         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1507         try:
1508             video_id = compute_video_id(local_video)
1509             was_reingest = db_instance.get_video(video_id) is not None
1510
1511             # Validate (with-context variant surfaces ffprobe_meta for the report)
1512             print("[CLI] Validating...")
1513             val_results, val_context = registry.run_all_with_context(
1514                 local_video, global_config
1515             )
1516             failures = [r for r in val_results if not r.passed]
1517
1518             # Save video status
1519             status = "failed" if failures else "processed"
1520             db_instance.add_video(
1521                 video_id, args.video_path, status, [r.model_dump() for r in val_results]
1522             )
1523
1524             seg_name = args.segmenter or getattr(

```

```

1525         getattr(global_config, "indexing", None), "default_segmenter", "motion"
1526     )
1527     segmenter_actual = None
1528     segmentation_ran = False
1529     try:
1530         print("\n" + "=" * 40)
1531         print("VALIDATION SUMMARY")
1532         print("=" * 40)
1533         for r in val_results:
1534             status_symbol = "[PASS]" if r.passed else "[FAIL]"
1535             print(f"{status_symbol} {r.validator_name}: {r.message}")
1536         print("=" * 40 + "\n")
1537
1538         if failures:
1539             print("[FAIL] Video failed checks. Indexing halted.")
1540             return
1541
1542         # Index
1543         segmenter_cls = segmenter_registry.get(seg_name)
1544         if not segmenter_cls:
1545             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1546             return
1547
1548         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1549         segmenter_actual = seg_name
1550         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1551         segmenter = segmenter_cls(db_instance, global_config)
1552         result = segmenter.process_video(
1553             video_id, local_video, max_frames=args.max_frames
1554         )
1555         segmentation_ran = True
1556
1557         if isinstance(result, Failure):
1558             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1559             print("[FAIL] Indexing halted for video.")
1560             segments = []
1561         else:
1562             segments = result.value if hasattr(result, "value") else result
1563             print(
1564                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
1565 segments inside database: {db_path}"
1566             )
1567             # Destroy-AFTER-confirm: keep new on success, restore prior on
1568 empty/failure.
1569         _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1570     finally:
1571         # Always emit the per-video observability report. Never raises.
1572         paths = emit_indexer_report(
1573             db=db_instance,
1574             video_id=video_id,
1575             # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
1576             # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
1577             # in DB provenance via db.add_video.
1578             video_path=local_video,
1579             config=_report_config_for_input(global_config, args.video_path),
1580             val_results=val_results,
1581             val_context=val_context,
1582             segmenter_requested=args.segmenter,
1583             segmenter_actual=segmenter_actual,
1584             was_reingest=was_reingest,
1585             segmentation_ran=segmentation_ran,
1586         )
1587     if paths:

```

```

1586             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1587             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1588             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1589         finally:
1590             storage.cleanup_acquired_local_input(
1591                 args.video_path, local_video, storage_cfg
1592             )
1593         return
1594
1595     # Start Local Server Mode
1596     if args.app or args.server or not args.video_path:
1597         if args.app:
1598             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1599             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1600             bind_host = "127.0.0.1"
1601             port = _resolve_app_port(bind_host, args.port)
1602             if port != args.port:
1603                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1604             url = f"http://{bind_host}:{port}/"
1605             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1606             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1607             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1608             print(
1609                 f"[APP] Starting KhelSutra ? opening {url} in your browser "
1610                 f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop."

```

14. assistant (2026-07-09T02:39:36.591Z)

This is the key CLI flow. I can see the checksum (1508), validation (1513), segmentation (1552), and report (1570) ? but notably no stitch step. Let me read the orchestration stitch functions, the compiler, and search for timing instrumentation.

15. user (2026-07-09T02:39:38.143Z)

```

1147     def _resolve_compile(
1148         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
1149     ):
1150         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
1151         row could change between submit and run). Verifies the video exists, the requested
segments match
1152         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
shot_count are exempt
1153         so manual/legacy picks can't silently vanish), and the output name is traversal-
safe. Raises
1154         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
safe_out_name)``."""
1155         video = db.get_video(video_id)
1156         if not video:
1157             raise _CompileError(404, "Indexed video record not found.")
1158         # Reject path traversal / absolute names (os.path.join would otherwise write
anywhere on disk).
1159         try:
1160             out_name = safe_output_filename(output_filename)
1161         except ValueError as e:
1162             raise _CompileError(400, str(e)) from e
1163
1164         all_segments = db.query_play_segments(video_id, {})
1165         matched = [s for s in all_segments if s["id"] in segment_ids]
1166         if not matched:

```

```

1167         raise _CompileError(400, "No matching play segments found to stitch.")
1168
1169     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1170     kept = []
1171     for s in matched:
1172         meta = s.get("metadata") or {}
1173         sc = meta.get("shot_count") if isinstance(meta, dict) else None
1174         try:
1175             if sc is not None and int(sc) < stitching.min_shot_count:
1176                 continue
1177             except (TypeError, ValueError):
1178                 pass # unparseable count -> treat as unknown, keep
1179             kept.append(s)
1180     if len(kept) < len(matched):
1181         logger.info(
1182             "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
floor.",
1183             stitching.min_shot_count,
1184             len(matched) - len(kept),
1185             len(matched),
1186         )
1187     if not kept:
1188         raise _CompileError(
1189             400,
1190             f"All {len(matched)} matching segments fall below "
1191             f"stitching.min_shot_count={stitching.min_shot_count}.",
1192         )
1193     return video, kept, out_name
1194
1195
1196 def reel_object_uri(cfg, video_id: str, name: str) -> str:
1197     """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
1198     (#463), or "" when
1199     the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
compile
1200     mirror-upload and the /api/highlights redirect:
1201     ``<output_root>/highlights/<video_id>/<name>``. """
1202     root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
1203     if not root or "://" not in root:
1204         return ""
1205     return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1206
1207
1208 def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1209     """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
1210     GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1211     isn't silently constructed as a fresh default. """
1212     sc = getattr(cfg, "storage", None)
1213     return sc.model_dump() if sc is not None else None
1214
1215
1216 def _rendition_encode_profile(stitching: Any, rendition: str) -> Optional[Any]:
1217     if rendition != "sd":
1218         return None
1219     renditions = (
1220         stitching.get("renditions")
1221         if isinstance(stitching, dict)
1222         else getattr(stitching, "renditions", None)
1223     )
1224     if isinstance(renditions, dict):
1225         return renditions.get("sd")
1226     return getattr(renditions, "sd", None)

```



```

1226
1227     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1228         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1229         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1230         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1231         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1232         reel_uri = reel_object_uri(cfg, video_id, name)
1233         if not reel_uri:
1234             return None
1235         settings = _storage_settings(cfg)
1236         if storage.exists(reel_uri, config=settings):
1237             return storage.signed_url(reel_uri, config=settings, method="GET")
1238         return None
1239
1240
1241     def _compile_wait_message(elapsed_sec: float) -> str:
1242         """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1243         ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
1244         minutes = int(elapsed_sec) // 60
1245         if minutes < 1:
1246             return "stitching on cloud GPU (NVENC) ? starting up"
1247         return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1248
1249
1250     def _stitch_reel(
1251         cfg,
1252         stitching,
1253         video_id: str,
1254         source_ref: str,
1255         time_pairs: List[Tuple[float, float]],
1256         out_name: str,
1257         locale: Optional[str],
1258         notify,
1259         rendition: str = "original",
1260         *,
1261         require_mirror: bool = False,
1262     ) -> Dict[str, Any]:
1263         """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1264         (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1265         needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1266         with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1267         trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1268         returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1269         ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure). The
1270         optional ``rendition`` selects the output encode profile; ``sd`` scales/transcodes
cheaply and
1271         never builds the original-size reel for that job.
1272
1273         ``require_mirror`` (the L4-worker caller sets True): on the worker the bucket mirror
is the ONLY
1274         delivery ? the worker's local reel is discarded when the Job exits, so a mirror-
upload failure or a
1275         missing bucket destination MUST fail the stitch (else the dispatcher reports success
for a reel the

```

```

1276     SPA can never download ? 404 on a "success" job). The control-plane caller keeps the
default False:
1277     its local reel is a valid fallback, so a mirror miss stays best-effort.""
1278     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1279
1280     rendition = rendition or "original"
1281     if rendition not in {"original", "sd"}:
1282         return {
1283             "status": "failed",
1284             "message": f"Unsupported compile rendition: {rendition}",
1285             "video_id": video_id,
1286         }
1287     output_dir = (
1288         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1289     )
1290     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1291     # unknown/absent locale keeps the configured default unchanged.
1292     localized_watermark = localize_watermark(stitching.watermark, locale)
1293     encode_profile = _rendition_encode_profile(stitching, rendition)
1294     compiler = _main.VideoCompiler(
1295         output_dir,
1296         begin_padding=stitching.begin_padding,
1297         end_padding=stitching.end_padding,
1298         watermark=localized_watermark,
1299         encoder=getattr(stitching, "encoder", "libx264"),
1300         single_pass=getattr(stitching, "single_pass_stitch", False),
1301         encode_profile=encode_profile,
1302     )
1303
1304     notify("stitching")
1305     logger.info(
1306         "[compile] stitching video_id=%s rendition=%s output=%s",
1307         video_id,
1308         rendition,
1309         out_name,
1310     )
1311     local_src = ""
1312     try:
1313         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1314         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1315         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1316         local_src = storage.acquire_local_input(
1317             source_ref, getattr(cfg, "storage", None)
1318         )
1319         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1320         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1321         with _LOCAL_COMPUTE_LANE:
1322             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1323     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1324         logger.error(
1325             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1326         )
1327     return {
1328         "status": "failed",
1329         "message": f"Highlight stitching failed: {e}",
1330         "video_id": video_id,
1331     }

```

```

1332         finally:
1333             if local_src:
1334                 storage.cleanup_acquired_local_input(
1335                     source_ref, local_src, getattr(cfg, "storage", None)
1336                 )
1337
1338             # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
1339             # URL ? Cloud Run
1340             # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
1341             # copy dies when the
1342             # instance recycles. On the control-plane a mirror-upload failure must NOT fail an
1343             # otherwise-good
1344             # stitch (the endpoint still falls back to the local file on the same instance). On
1345             # the L4 worker
1346             # (`require_mirror`) the mirror is the ONLY delivery ? the worker's local reel is
1347             # discarded when
1348             # the Job exits ? so a mirror failure (or a missing bucket destination) MUST fail
1349             # the stitch, else
1350             # the dispatcher would report success for a reel the SPA can never download (a 404
1351             # on a "success").
1352             reel_uri = reel_object_uri(cfg, video_id, out_name)
1353             if not reel_uri and require_mirror:
1354                 logger.error(
1355                     "[compile] worker stitch has no bucket destination for video_id=%s ?
1356                     storage.output_root "
1357                     "must be a bucket URI (gs://?) on the cloud compile path; failing the
1358                     reel.",
1359                     video_id,
1360                 )
1361             return {
1362                 "status": "failed",
1363                 "message": (
1364                     "cloud compile has no bucket output ? set storage.output_root to a
1365                     bucket "
1366                     "(gs://?) so the reel can be delivered."
1367                 ),
1368                 "video_id": video_id,
1369             }
1370         if reel_uri:
1371             try:
1372                 notify("uploading reel")
1373                 # Thread the active storage settings so the configured backend (S3
1374                 endpoint/region, GCS
1375                 # project/client) is used, not a fresh default.
1376                 storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1377             except Exception as e: # noqa: BLE001 ? see require_mirror above
1378                 if require_mirror:
1379                     # No local fallback on the worker ? a lost mirror IS a lost reel; fail
1380                     # honestly so the
1381                     # dispatcher surfaces a clean failed job the user can retry (not a
1382                     # 404-on-success).
1383                     logger.error(
1384                         "[compile] worker reel mirror-upload to %s FAILED for video_id=%s ?
1385                         failing the "
1386                         "stitch (no local fallback on the L4 worker): %s",
1387                         reel_uri,
1388                         video_id,
1389                         e,
1390                         exc_info=True,
1391                     )
1392                 return {
1393                     "status": "failed",
1394                     "message": f"reel upload to {reel_uri} failed: {e}",
1395                     "video_id": video_id,
1396                 }

```

```

1383         logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1384
1385         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1386         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1387         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1388         notify("finished")
1389         return {
1390             "status": "success",
1391             "filepath": out_path,
1392             "download_url": download_url,
1393             "video_id": video_id,
1394             "reel_uri": reel_uri or "",
1395             "filename": out_name,
1396             "rendition": rendition,
1397         }
1398
1399
1400     def _dispatch_compile_remote(
1401         cfg,
1402         db,
1403         video_id: str,
1404         segment_ids: List[int],
1405         output_filename: str,
1406         locale: Optional[str],
1407         notify,
1408         rendition: str = "original",
1409     ) -> Dict[str, Any]:
1410         """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
1411         control-plane owns the DB, so it RESOLVES the reel's segments here
(`_resolve_compile`), stages a
1412         small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
1413         to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
1414         done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
1415         worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
1416         ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
1417         from backend.compute import CompileJobSpec, get_compute_backend
1418
1419         notify("resolving")
1420         try:
1421             video, kept, out_name = _resolve_compile(
1422                 db, cfg, video_id, segment_ids, output_filename
1423             )
1424         except _CompileError as e:
1425             return {"status": "failed", "message": e.detail, "video_id": video_id}
1426

```

16. user (2026-07-09T02:39:40.463Z)

```

Found 32 files
backend\orchestration.py
backend\main.py
backend\worker\__main__.py
backend\config\models.py
backend\api\job_worker.py
backend\compute\base.py
backend\pipeline\validators\base.py
backend\infrastructure\database_media.py
backend\infrastructure\database.py
backend\ui\assets\index-kJh4dVE.js
backend\pipeline\validators\frame_sampler.py

```

```

backend\ui\assets\index-BjemzRyy.css
backend\pipeline\detectors\wasb_infer.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\infrastructure\execution_policy.py
backend\eval\fusion_golden.py
backend\server_runtime.py
backend\pipeline\detectors\base.py
backend\pipeline\detectors\tracknet_runner.py
backend\utils\run_telemetry.py
backend\sports\badminton.py
backend\providers\gemini.py
backend\reporting\__init__.py
backend\reporting\harvest.py
backend\pipeline\segmenters\motion.py
backend\pipeline\detectors\fusion_features.py
backend\eval\tolerance_metrics.py
backend\api\auth.py
backend\eval\gemini_refine.py
backend\reporting\spec.yaml

```

17. assistant (2026-07-09T02:39:49.871Z)

Key finding so far: the CLI single-shot flow does checksum ? validate ? segment ? report but no stitch. There's a `backend/utils/run\_telemetry.py` that looks central to timing. Let me read the telemetry, reporting, the `/api/compile` handler, and `\_execute\_compile`.

18. user (2026-07-09T02:39:50.888Z)

```

1      """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3      A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
4      die with
5      **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
6      event). When
7      that happens the only forensic evidence is whatever was last flushed to disk. This
8      module keeps a
9      tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
10     is a
11     bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
12     4.8 GB VRAM
13     free, CPU 60 %, RAM 21/32 GB ? then vanished."
14
15     It is the **observability** companion to the `ExecutionPolicy` *control* layer
16     (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
17     signals
18     that explain *why* a run slowed or died, and lets us compare execution velocity across
19     quality
20     iterations.
21
22     Design:
23     - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
24     from an
25     existing progress callback; fps is derived from the delta since the previous snapshot.
26     ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
27     sample, so
28     the last line before a kill captures the machine state at death.
29     - **Capped + rotating** : a fixed ``meta`` header line (machine / GPU / start) is always
30     kept; recent
31     snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
32     is rewritten
33     atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
34     written.
35     - **Graceful degradation** : GPU stats come from ``nvidia-smi`` (absent -> ``None``);

```

```

system stats
24     from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25     hiccup must not break the run it observes.
26     - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27         and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29     All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30     ""
31
32     from __future__ import annotations
33
34     import json
35     import os
36     import shutil
37     import subprocess
38     import time
39     from typing import Any, Callable, Dict, List, Optional
40
41     GpuFn = Callable[[], Optional[Dict[str, Any]]]
42     SysFn = Callable[[], Optional[Dict[str, Any]]]
43
44     # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
45     _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
46     _THROTTLE_CLOCK_FRAC = 0.8 # SM clock below this fraction of max => flagged throttled
47     _THROTTLE_TEMP_C = (
48         84.0 # ... or temperature at/above this (laptop Max-Q throttle territory)
49     )
50
51
52     def _num(s: str) -> Optional[float]:
53         try:
54             return float(s)
55         except (TypeError, ValueError):
56             return None
57
58
59     def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
60         """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
61
62         Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
63         derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
64         ""
65         exe = shutil.which("nvidia-smi")
66         if not exe:
67             return None
68         try:
69             out = subprocess.run(
70                 [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
71                 capture_output=True,
72                 text=True,
73                 timeout=timeout,
74                 check=True,
75             ).stdout.strip()
76         except (subprocess.SubprocessError, OSError):
77             return None
78         row = out.splitlines()[0] if out else ""
79         parts = [p.strip() for p in row.split(",")]
80         if len(parts) < 7:

```

```

81         return None
82     vals = [_num(p) for p in parts[:7]]
83     temp, util, power, sm, sm_max, used, free = vals
84     throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
85     if sm is not None and sm_max:
86         throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
87     return {
88         "util_pct": util,
89         "temp_c": temp,
90         "power_w": power,
91         "sm_mhz": sm,
92         "sm_max_mhz": sm_max,
93         "vram_used_mb": used,
94         "vram_free_mb": free,
95         "throttled": throttled,
96     }
97
98
99     def system_stats() -> Optional[Dict[str, Any]]:
100         """CPU % + RAM via `psutil`, or `None` if psutil isn't installed. Never
101         raises."""
102         try:
103             import psutil # optional dep (BSD-3); only used when present
104         except ImportError:
105             return None
106         try:
107             vm = psutil.virtual_memory()
108             return {
109                 "cpu_pct": psutil.cpu_percent(interval=None),
110                 "ram_used_gb": round(vm.used / 1e9, 2),
111                 "ram_total_gb": round(vm.total / 1e9, 2),
112                 "ram_pct": vm.percent,
113             }
114         except Exception: # noqa: BLE001 - psutil edge cases must not break the run
115             return None
116
117     def gpu_name() -> Optional[str]:
118         """The GPU model string (for the runlog header), or `None` without nvidia-smi."""
119         exe = shutil.which("nvidia-smi")
120         if not exe:
121             return None
122         try:
123             out = subprocess.run(
124                 [exe, "--query-gpu=name", "--format=csv,noheader"],
125                 capture_output=True,
126                 text=True,
127                 timeout=4.0,
128                 check=True,
129             ).stdout
130         except (subprocess.SubprocessError, OSError):
131             return None
132         return out.splitlines()[0].strip() if out else None
133
134
135     class RunTelemetry:
136         """A small rotating JSONL runlog of execution velocity + machine health.
137
138         Call `snapshot(done, total, phase=...)` from a progress callback and `event(kind,
139         msg)` at
140         milestones. The file holds a `meta` header line plus the most recent snapshot /
141         event lines
142         that fit within `max_bytes` (~10 KB), rewritten atomically on every write.
143         """

```

```

143     def __init__(
144         self,
145         path: str,
146         label: str = "run",
147         max_bytes: int = 10_000,
148         gpu_fn: GpuFn = gpu_stats,
149         sys_fn: SysFn = system_stats,
150         clock: Callable[[], float] = time.time,
151     ) -> None:
152         self.path = path
153         self.label = label
154         self.max_bytes = max_bytes
155         self._gpu_fn = gpu_fn
156         self._sys_fn = sys_fn
157         self._clock = clock
158         self._t0 = clock()
159         self._last_done: Optional[int] = None
160         self._last_t: Optional[float] = None
161         self._lines: List[
162             str
163         ] = [] # recent body lines (on-disk ring, trimmed to max_bytes)
164         self._meta = json.dumps(
165             {
166                 "meta": {
167                     "label": label,
168                     "started": self._iso(self._t0),
169                     "pid": os.getpid(),
170                     "gpu_name": gpu_name(),
171                     "cpu_count": os.cpu_count(),
172                 }
173             },
174             separators=(",", ":"),
175         )
176         self._write()
177
178     # -- public API -----
179     def snapshot(
180         self,
181         done: Optional[int] = None,
182         total: Optional[int] = None,
183         phase: Optional[str] = None,
184         **extra: Any,
185     ) -> Dict[str, Any]:
186         """Record a velocity + health sample; ``fps`` is derived from the previous
187         snapshot."""
188         now = self._clock()
189         rec: Dict[str, Any] = {
190             "t": self._iso(now),
191             "elapsed_s": round(now - self._t0, 1),
192         }
193         if phase is not None:
194             rec["phase"] = phase
195         if done is not None:
196             rec["done"] = done
197             if total:
198                 rec["total"] = total
199                 rec["pct"] = round(100.0 * done / total, 1)
200             if (
201                 self._last_done is not None
202                 and self._last_t is not None
203                 and now > self._last_t
204             ):
205                 rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
206                 self._last_done, self._last_t = done, now
207         rec.update(extra)

```



```

207         self._attach_health(rec)
208         self._append(rec)
209         return rec
210
211     def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
212         """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
213         now = self._clock()
214         rec: Dict[str, Any] = {
215             "t": self._iso(now),
216             "elapsed_s": round(now - self._t0, 1),
217             "event": kind,
218         }
219         if msg:
220             rec["msg"] = msg
221             rec.update(extra)
222             self._attach_health(rec)
223             self._append(rec)
224             return rec
225
226     # -- internals -----
227     def _attach_health(self, rec: Dict[str, Any]) -> None:
228         gpu = self._safe(self._gpu_fn)
229         sys_ = self._safe(self._sys_fn)
230         if gpu is not None:
231             rec["gpu"] = gpu
232         if sys_ is not None:
233             rec["sys"] = sys_
234
235     def _append(self, rec: Dict[str, Any]) -> None:
236         self._lines.append(json.dumps(rec, separators=(",", ":")))
237         budget = self.max_bytes - len(self._meta) - 1
238         while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
239             self._lines.pop(0) # drop oldest body line; the meta header is always kept
240         self._write()
241
242     def _write(self) -> None:
243         try:
244             tmp = self.path + ".tmp"
245             with open(tmp, "w", encoding="utf-8") as f:
246                 f.write(self._meta + "\n")
247                 for s in self._lines:
248                     f.write(s + "\n")
249             os.replace(tmp, self.path)
250         except OSError:
251             pass # telemetry must never break the run it observes
252
253     @staticmethod
254     def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
255         try:
256             return fn()
257         except Exception: # noqa: BLE001 - a collector hiccup must not break the run
258             return None
259
260     @staticmethod
261     def _iso(t: float) -> str:
262         return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
263

```

19. user (2026-07-09T02:39:51.767Z)

```

1559     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1560         """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
1561         the ``{status, filepath, download_url}`` result the SPA polls for. Returns a

```

```

1562     ``status='failed'``
1563     result for an EXPECTED problem (missing video / no segments / a stitch error) so the
1564     SPA always gets
1565     a clean verdict instead of a worker-crash error.
1566     #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
1567     and cloud
1568     dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
1569     (``dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
1570     reel on the
1571     2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
1572     here, unchanged.````
1573     notify = progress or (lambda stage: None)
1574     params = json.loads(job.get("params") or "{}")
1575     video_id = job.get("video_id") or ""
1576     segment_ids = params.get("segment_ids") or []
1577     locale = params.get("locale")
1578     output_filename = params.get("output_filename", "")
1579     rendition = str(params.get("rendition") or "original")
1580     if rendition not in {"original", "sd"}:
1581         return {
1582             "status": "failed",
1583             "message": f"Unsupported compile rendition: {rendition}",
1584             "video_id": video_id,
1585         }
1586     if (
1587         resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
1588         and _cloud_available(cfg)
1589     ):
1590         return _dispatch_compile_remote(
1591             cfg, db, video_id, segment_ids, output_filename, locale, notify, rendition
1592         )
1593     notify("resolving")
1594     try:
1595         video, kept, out_name = _resolve_compile(
1596             db, cfg, video_id, segment_ids, output_filename
1597         )
1598     except _CompileError as e:
1599         return {"status": "failed", "message": e.detail, "video_id": video_id}
1600     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1601     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1602     return _stitch_reel(
1603         cfg,
1604         stitching,
1605         video_id,
1606         video["filepath"],
1607         time_pairs,
1608         out_name,
1609         locale,
1610         notify,
1611         rendition,
1612     )

```

20. assistant (2026-07-09T02:40:05.323Z)

Now let me find the `/api/compile` handler in main.py, check where RunTelemetry is actually wired in, and read the reporting module that emits the per-video report.

21. user (2026-07-09T02:40:08.306Z)

```
70:from backend.reporting import emit_indexer_report
```

```

480:     NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ? the
engine
512:     Mirrors /api/highlights' safety (bare-name + containment checks)."""
625:     # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
1040:@app.get("/api/highlights/{video_id}/{filename}")
1042:     """Stream a compiled highlight reel ? `filename` is the bare name given to /api/compile
1075:         detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1097:@app.post("/api/compile", status_code=202)
1098:def compile_highlights(req: CompileRequest, request: Request):
1429:     # /api/compile ? reads the same value instead of a write-only runtime hack.
1570:         paths = emit_indexer_report(

```

22. user (2026-07-09T02:40:09.900Z)

```

backend\main.py:70:from backend.reporting import emit_indexer_report
backend\main.py:1570:         paths = emit_indexer_report(
backend\orchestration.py:16:``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``,
``VideoCompiler``.
backend\orchestration.py:1093:         paths = _main.emit_indexer_report(
backend\worker\_main_.py:40:from backend.reporting import emit_indexer_report
backend\worker\_main_.py:381:         emit_indexer_report(
backend\pipeline\detectors\base.py:59:segmenter Results (or pass detector_health into
emit_indexer_report context), the
backend\eval\fusion_golden.py:33:from backend.utils.run_telemetry import RunTelemetry
backend\eval\fusion_golden.py:176:     `telemetry` (a `RunTelemetry` or None) records per-frame
velocity + machine-health snapshots
backend\eval\fusion_golden.py:216:         rec = telemetry.snapshot(done, total,
phase=spec["name"])
backend\eval\fusion_golden.py:320:     telem: Optional[RunTelemetry]
backend\eval\fusion_golden.py:322:         telem = RunTelemetry(
backend\eval\fusion_golden.py:323:             os.path.join(out_dir, "run_telemetry.jsonl"),
label="fusion_golden"
backend\eval\fusion_golden.py:384:         telem.event("video_start", spec["name"])
backend\eval\fusion_golden.py:397:         telem.event("error", spec["name"], err=str(e))
backend\eval\fusion_golden.py:403:         telem.event(
backend\utils\run_telemetry.py:135:class RunTelemetry:
backend\reporting\run_signals.py:6:by video_id; `emit_indexer_report` pops them when the run's
report is built.
backend\reporting\__init__.py:5:`emit_indexer_report(...)` is designed to be called from a
`finally` block and
backend\reporting\__init__.py:29:def emit_indexer_report(

```

23. assistant (2026-07-09T02:40:17.831Z)

Critical finding: `RunTelemetry` is only wired into `backend/eval/fusion\_golden.py`, not the main CLI/orchestration pipeline. Let me read the `/api/compile` handler, the reporting module, and the reporting harvest to see what timing the per-video report captures.

24. user (2026-07-09T02:40:19.888Z)

```

1097     @app.post("/api/compile", status_code=202)
1098     def compile_highlights(req: CompileRequest, request: Request):
1099         """Submit a reel-COMPIL job (202 + job_id); poll GET /api/jobs/{job_id} for the
reel (#329).
1100
1101         Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
concat + watermark)
1102         can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
synchronously returned a
1103         Cloudflare 524 while the engine kept going (the reel built, but the client saw
"failed" and never
1104         got the URL). The cheap checks (video exists, segments match + survive the floor,
safe filename)
1105         fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's

```

```

`result` carries
1106     {filepath, download_url} ? the same shape the sync endpoint returned, now under
/api/jobs/{id}."""
1107     # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
before queuing.
1108     try:
1109         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1110     except edge_auth.EdgeAuthError as e:
1111         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1112     try:
1113         video, _kept, out_name = _resolve_compile(
1114             request.app.state.db,
1115             request.app.state.config,
1116             req.video_id,
1117             req.segment_ids,
1118             req.output_filename,
1119         )
1120     except _CompileError as e:
1121         raise HTTPException(status_code=e.status_code, detail=e.detail) from e
1122
1123     job_id = uuid.uuid4().hex
1124     # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
the row by id so
1125     # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
localized reel.
1126     if not request.app.state.db.create_compile_job(
1127         job_id,
1128         req.video_id,
1129         video["filepath"],
1130         req.segment_ids,
1131         out_name,
1132         locale=req.locale,
1133         owner_id=owner_id,
1134         rendition=req.rendition,
1135     ):
1136         raise HTTPException(
1137             status_code=500, detail="Failed to persist compile job record."
1138         )
1139     _get_executor().submit(
1140         _drain_due_jobs, request.app.state.config, request.app.state.db
1141     )
1142     return JSONResponse(
1143         status_code=202,
1144         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1145     )
1146
1147
1148 # --- CLI Debug Utility & Orchestration ---
1149 def run_cli_diagnose_clip(video_path: str, timestamp: float):
1150     """CLI debugging utility to examine segment properties around a timestamp."""
1151     print("=" * 60)

```

25. user (2026-07-09T02:40:20.378Z)

```

1     """Per-video observability reports for the indexer.
2
3     `obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
4     no-op and the pipeline runs exactly as before. The single entrypoint
5     `emit_indexer_report(...)` is designed to be called from a `finally` block and
6     NEVER raises ? emitting a report must not be able to break a processing run.
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, Dict, List, Optional

```

```

12
13 from . import run_signals # no obsreport dependency ? always importable
14
15 try: # soft dependency ? never hard-fail the pipeline on a reporting import
16     import obsreport # noqa: F401
17
18     from . import harvest
19
20     OBSREPORT_AVAILABLE = True
21 except Exception: # pragma: no cover - exercised only when obsreport is absent
22     OBSREPORT_AVAILABLE = False
23
24
25 def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
26     return bool((config or {}).get("report", {}).get("enabled", True))
27
28
29 def emit_indexer_report(
30     *,
31     db,
32     video_id: str,
33     video_path: Optional[str],
34     config: Optional[Dict[str, Any]],
35     val_results: List[Any],
36     val_context: Optional[Dict[str, Any]] = None,
37     segmenter_requested: Optional[str] = None,
38     segmenter_actual: Optional[str] = None,
39     was_reingest: bool = False,
40     segmentation_ran: bool = True,
41 ) -> Optional[Dict[str, str]]:
42     """Harvest + write the report. Returns the written file paths, or None if
43     reporting is unavailable/disabled/failed. Never raises."""
44     config = config or {}
45     # Accept either a plain dict or a typed config object (e.g.
46     backend.config.AppConfig).
47     # Normalize to a plain nested dict so harvest's dict access works uniformly.
48     if hasattr(config, "model_dump"):
49         try:
50             config = config.model_dump(mode="json")
51         except Exception:
52             config = {}
53     # Consume run signals unconditionally (even when reporting is off) so the
54     # process-local store never accumulates entries for finished runs.
55     signals = run_signals.pop(video_id)
56     if not OBSREPORT_AVAILABLE or not report_enabled(config):
57         return None
58     try:
59         play_segments = db.query_play_segments(video_id, {}) if db is not None else []
60         candidates = db.query_candidate_segments(video_id) if db is not None else []
61         rec = harvest.record_run(
62             video_id=video_id,
63             video_path=video_path,
64             config=config,
65             val_results=val_results,
66             val_context=val_context,
67             play_segments=play_segments,
68             candidates=candidates,
69             segmenter_requested=segmenter_requested,
70             segmenter_actual=segmenter_actual,
71             was_reingest=was_reingest,
72             segmentation_ran=segmentation_ran,
73             run_signals=signals,
74         )
75     output_dir = (config.get("report") or {}).get("output_dir")
76     res = rec.write(output_dir=output_dir)

```

```
76         return res.paths or None
77     except Exception: # pragma: no cover - defensive; reporting must never break a run
78         return None
79
```

26. user (2026-07-09T02:40:22.180Z)

```
63:     if fmt.get("duration"):
65:         meta["duration_s"] = round(float(fmt["duration"]), 1)
111:         total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
186:     total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
188:     dur = video_meta.get("duration_s")
```

27. assistant (2026-07-09T02:40:33.860Z)

The report captures video content durations, not wall-clock. Let me check `run_signals` and `spec.yaml` for any processing-time capture, and start mapping the segmenters (GPU/cloud) and the eval/regression harnesses.

28. user (2026-07-09T02:40:35.369Z)

```
1     """In-process signal channel: pipeline internals -> the per-video report.
2
3     Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
4     deep inside a segmenter's worker threads, with no return path to the harvest ?
5     segmenters return only their segment list. Layers record those facts here keyed
6     by video_id; `emit_indexer_report` pops them when the run's report is built.
7
8     Process-local and best-effort BY DESIGN: recording must never break a run, the
9     store is consumed (popped) once per report, and signals for a video are simply
10    dropped if no report is emitted. No obsreport dependency ? importable everywhere.
11    """
12
13    from __future__ import annotations
14
15    import threading
16    from typing import Any, Dict
17
18    _lock = threading.Lock()
19    _signals: Dict[str, Dict[str, Any]] = {}
20
21
22    def incr(video_id: str, key: str, by: int = 1) -> None:
23        """Add `by` to the counter `key` for this video's in-flight run."""
24        if not video_id or by == 0:
25            return
26        with _lock:
27            sig = _signals.setdefault(video_id, {})
28            sig[key] = int(sig.get(key, 0)) + by
29
30
31    def pop(video_id: str) -> Dict[str, Any]:
32        """Consume (remove and return) all signals recorded for this video's run."""
33        with _lock:
34            return _signals.pop(video_id, {})
35
```

29. user (2026-07-09T02:40:36.432Z)

```
1     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
2     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
3     from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
4     from backend.pipeline.segmenters.motion import MotionPlaySegmenter
5     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
```

```

6      from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
7      from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
8
9      __all__ = [
10         "segmenter_registry",
11         "BasePlaySegmenter",
12         "MotionPlaySegmenter",
13         "GeminiPlaySegmenter",
14         "HybridYoloSegmenter",
15         "TrackNetHybridSegmenter",
16         "WasbHybridSegmenter",
17         "FusionSegmenter",
18     ]
19

```

30. user (2026-07-09T02:40:40.989Z)

78: # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
85: was used. Because rally timing and motion thresholds scale with fps, boundaries

31. assistant (2026-07-09T02:40:48.257Z)

Confirmed: the report layer captures counters and content durations only ? no wall-clock/per-phase latency. Now let me map segmenter registration names and their GPU/cloud dependencies.

32. user (2026-07-09T02:40:49.884Z)

```

1      from abc import ABC, abstractmethod
2      from typing import Any, Dict, List, Optional, Type
3
4      from backend.infrastructure.database import Database
5      from backend.results import (
6          Result, # new standardized contract (Result[Value] == Success[Value] | Failure)
7      )
8
9
10     def _normalize_config(config: Any) -> Any:
11         """Normalize a plain-dict config into an AppConfig so segmenters can use typed
12         attribute access consistently. Production always passes AppConfig; tests may
13         pass plain dicts ? this bridge keeps both paths working."""
14         if isinstance(config, dict):
15             from backend.config.models import AppConfig
16
17             try:
18                 return AppConfig(**config)
19             except Exception:
20                 return (
21                     config # invalid config ? let the segmenter's own validation reject it
22                 )
23         return config
24
25
26     class BasePlaySegmenter(ABC):
27         def __init__(self, db: Database, config: Dict[str, Any]):
28             self.db = db
29             self.config = _normalize_config(config)
30
31         @property
32         @abstractmethod
33         def name(self) -> str:
34             """Unique identifier for this segmenter."""
35             pass
36
37         @property

```

```

38     @abstractmethod
39     def description(self) -> str:
40         """Human-readable explanation of this segmenter."""
41         pass
42
43     @abstractmethod
44     def process_video(
45         self,
46         video_id: str,
47         video_path: str,
48         player_pool: List[str] = None,
49         max_frames: int = None,
50     ) -> "Result[List[Dict[str, Any]]]":
51         """
52         Processes a video to detect play segments, populates SQLite database,
53         and returns a Result.
54
55         Use isinstance(result, Failure) (or result.is_success()) to distinguish
56         "no segments found" from "something went wrong" (missing key, healthcheck
57         failure, etc.). This is the start of the standardized error/result contract.
58         """
59         pass
60
61
62     class SegmenterRegistry:
63         def __init__(self):
64             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}
65
66         def register(self, segmenter_cls: Type[BasePlaySegmenter]):
67             """Registers a new segmenter class under its name property."""
68             try:
69                 temp_inst = segmenter_cls(None, {})
70                 name = temp_inst.name
71             except Exception as e:
72                 raise ValueError(
73                     f"Failed to resolve name property from segmenter class
74 {segmenter_cls.__name__}: {e}"
75                 ) from e
76             self._segmenters[name] = segmenter_cls
77
78         def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
79             return self._segmenters.get(name)
80
81         def list_available(self) -> Dict[str, str]:
82             """Returns a dict of registered segmenter names and their descriptions."""
83             res = {}
84             for name, cls in self._segmenters.items():
85                 try:
86                     temp_inst = cls(None, {})
87                     desc = temp_inst.description
88                 except Exception:
89                     desc = "No description available."
90             res[name] = desc
91             return res
92
93     segmenter_registry = SegmenterRegistry()
94

```

33. user (2026-07-09T02:40:51.856Z)

```

backend\pipeline\segmenters\fusion_hybrid.py:41:class
FusionSegmenter(TrajectoryHybridSegmenter):
backend\pipeline\segmenters\fusion_hybrid.py:243:segmenter_registry.register(FusionSegmenter)
backend\pipeline\segmenters\motion.py:14:class MotionPlaySegmenter(BasePlaySegmenter):

```



```

backend\pipeline\segmenters\motion.py:243:segmenter_registry.register(MotionPlaySegmenter)
backend\pipeline\segmenters\gemini.py:75:class GeminiPlaySegmenter(BasePlaySegmenter):
backend\pipeline\segmenters\gemini.py:95:         sport_name = self.config.sport
backend\pipeline\segmenters\gemini.py:103:         provider_name = self.config.ai_provider
backend\pipeline\segmenters\gemini.py:633:         model_name = self.config.ai_model
backend\pipeline\segmenters\gemini.py:707:segmenter_registry.register(GeminiPlaySegmenter)
backend\pipeline\segmenters\base.py:26:class BasePlaySegmenter(ABC):
backend\pipeline\segmenters\base.py:62:class SegmenterRegistry:
backend\pipeline\segmenters\base.py:66:     def register(self, segmenter_cls:
Type[BasePlaySegmenter]):
backend\pipeline\segmenters\base.py:70:         name = temp_inst.name
backend\pipeline\segmenters\base.py:81:         """Returns a dict of registered segmenter names
and their descriptions."""
backend\pipeline\segmenters\trajectory_hybrid.py:18:This base is deliberately NOT registered;
only concrete subclasses register.
backend\pipeline\segmenters\trajectory_hybrid.py:50:class
TrajectoryHybridSegmenter(BasePlaySegmenter):
backend\pipeline\segmenters\trajectory_hybrid.py:190:         sport_name = self.config.sport
backend\pipeline\segmenters\trajectory_hybrid.py:200:         provider_name =
self.config.ai_provider
backend\pipeline\segmenters\trajectory_hybrid.py:202:         model_name = "local-noai"
backend\pipeline\segmenters\trajectory_hybrid.py:210:         model_name =
self.config.ai_model
backend\pipeline\segmenters\wasb_hybrid.py:25:class
WasbHybridSegmenter(TrajectoryHybridSegmenter):
backend\pipeline\segmenters\wasb_hybrid.py:68:segmenter_registry.register(WasbHybridSegmenter)
backend\pipeline\segmenters\_keyhint.py:12:     if os.name == "nt": # Windows
backend\pipeline\segmenters\yolo_hybrid.py:48:class HybridYoloSegmenter(BasePlaySegmenter):
backend\pipeline\segmenters\yolo_hybrid.py:75:         sport_name = self.config.sport
backend\pipeline\segmenters\yolo_hybrid.py:83:         provider_name = self.config.ai_provider
backend\pipeline\segmenters\yolo_hybrid.py:84:         model_name = self.config.ai_model
backend\pipeline\segmenters\yolo_hybrid.py:507:         provider_name = self.config.ai_provider
backend\pipeline\segmenters\yolo_hybrid.py:508:         model_name = self.config.ai_model
backend\pipeline\segmenters\yolo_hybrid.py:596:segmenter_registry.register(HybridYoloSegmenter)
backend\pipeline\segmenters\tracknet_hybrid.py:23:class
TrackNetHybridSegmenter(TrajectoryHybridSegmenter):
backend\pipeline\segmenters\tracknet_hybrid.py:48:segmenter_registry.register(TrackNetHybridSegm
enter)

```

34. user (2026-07-09T02:40:53.891Z)

```

backend\pipeline\segmenters\gemini.py:106:         api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\gemini.py:107:         api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\gemini.py:108:         if not api_key:
backend\pipeline\segmenters\gemini.py:109:             from backend.pipeline.segmenters._keyhint
import api_key_hint
backend\pipeline\segmenters\gemini.py:112:         f"{api_key_env_var} is not set ? the
{provider_name} segmenter cannot run. "
backend\pipeline\segmenters\gemini.py:113:         f"Set your API key. " +
api_key_hint(api_key_env_var)
backend\pipeline\segmenters\gemini.py:122:         provider_name, api_key=api_key,
model_name=model_name
backend\pipeline\segmenters\gemini.py:364:         "             GEMINI CHUNKING SUMMARY",
backend\pipeline\segmenters\gemini.py:431:         "             GEMINI SINGLE FILE SUMMARY",
backend\pipeline\segmenters\fusion_hybrid.py:4:TrackNet optional) to seed candidate windows
exactly as `wasb_hybrid`/`tracknet_hybrid`
backend\pipeline\segmenters\fusion_hybrid.py:24:and hands off identically to `wasb_hybrid`.
Registered as ``fusion_hybrid``, NOT the default
backend\pipeline\segmenters\fusion_hybrid.py:32:from backend.pipeline.detectors import
fusion_features as ff
backend\pipeline\segmenters\fusion_hybrid.py:33:from backend.pipeline.detectors import
rally_gate
backend\pipeline\segmenters\fusion_hybrid.py:34:from backend.pipeline.detectors.base import
DetectorRunner

```

```

backend\pipeline\segmenters\fusion_hybrid.py:35:from backend.pipeline.detectors.tracknet_runner
import TrackNetConfig, TrackNetRunner
backend\pipeline\segmenters\fusion_hybrid.py:36:from backend.pipeline.detectors.wasb_runner
import WasbConfig, WasbRunner
backend\pipeline\segmenters\fusion_hybrid.py:74:         if substrate == "tracknet":
backend\pipeline\segmenters\fusion_hybrid.py:79:             "fusion_substrate": "tracknet",
backend\pipeline\segmenters\fusion_hybrid.py:84:             "fusion_substrate": "wasb",
backend\pipeline\segmenters\fusion_hybrid.py:90:         # Native (no-WSL) path supports the WASB
substrate only ? there is no native TrackNet
backend\pipeline\segmenters\fusion_hybrid.py:93:         if substrate != "wasb":
backend\pipeline\segmenters\fusion_hybrid.py:96:
"indexing.fusion.substrate='wasb' (no native TrackNet runner yet)."
backend\pipeline\segmenters\fusion_hybrid.py:98:         from
backend.pipeline.detectors.native_wasb_runner import (
backend\pipeline\segmenters\fusion_hybrid.py:107:             "fusion_substrate": "wasb",
backend\pipeline\segmenters\fusion_hybrid.py:108:             "device": cfg.device,
backend\pipeline\segmenters\fusion_hybrid.py:176:         from
backend.pipeline.detectors.person_detector import PersonDetector
backend\pipeline\segmenters\tracknet_hybrid.py:4:in WSL2
(backend/pipeline/detectors/tracknet_runner.py, ADR-001). Rationale: a
backend\pipeline\segmenters\tracknet_hybrid.py:17:from backend.pipeline.detectors.base import
DetectorRunner
backend\pipeline\segmenters\tracknet_hybrid.py:18:from
backend.pipeline.detectors.tracknet_runner import TrackNetConfig, TrackNetRunner
backend\pipeline\segmenters\tracknet_hybrid.py:24:     family = "tracknet"
backend\pipeline\segmenters\tracknet_hybrid.py:29:         return "tracknet_hybrid"
backend\pipeline\segmenters\tracknet_hybrid.py:34:         "Two-stage pipeline: TrackNetV4
shuttle-trajectory detection (WSL2) for precise "
backend\pipeline\segmenters\wasb_hybrid.py:4:WSL2 (backend/pipeline/detectors/wasb_runner.py).
WASB ships pretrained
backend\pipeline\segmenters\wasb_hybrid.py:19:from backend.pipeline.detectors.base import
DetectorRunner
backend\pipeline\segmenters\wasb_hybrid.py:20:from backend.pipeline.detectors.wasb_runner import
WasbConfig, WasbRunner
backend\pipeline\segmenters\wasb_hybrid.py:26:     family = "wasb"
backend\pipeline\segmenters\wasb_hybrid.py:31:         return "wasb_hybrid"
backend\pipeline\segmenters\wasb_hybrid.py:36:         "Two-stage pipeline: WASB shuttle-
trajectory detection (WSL2, pretrained badminton "
backend\pipeline\segmenters\wasb_hybrid.py:51:         from
backend.pipeline.detectors.native_wasb_runner import (
backend\pipeline\segmenters\wasb_hybrid.py:60:             "device": cfg.device,
backend\pipeline\segmenters\_keyhint.py:10:def api_key_hint(var: str) -> str:
backend\pipeline\segmenters\_keyhint.py:13:         return f'In PowerShell:
$env:{var}="your_api_key_here"'
backend\pipeline\segmenters\_keyhint.py:14:     return f"Set it with: export
{var}=your_api_key_here"
backend\pipeline\segmenters\__init__.py:5:from backend.pipeline.segmenters.tracknet_hybrid
import TrackNetHybridSegmenter
backend\pipeline\segmenters\__init__.py:6:from backend.pipeline.segmenters.wasb_hybrid import
WasbHybridSegmenter
backend\pipeline\segmenters\yolo_hybrid.py:27:from backend.pipeline.detectors.person_detector
import PersonDetector
backend\pipeline\segmenters\yolo_hybrid.py:87:         api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\yolo_hybrid.py:88:         api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:89:         if not api_key:
backend\pipeline\segmenters\yolo_hybrid.py:90:             from
backend.pipeline.segmenters._keyhint import api_key_hint
backend\pipeline\segmenters\yolo_hybrid.py:93:             f"{api_key_env_var} environment
variable is not set! "
backend\pipeline\segmenters\yolo_hybrid.py:95:             + api_key_hint(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:102:             provider_name, api_key=api_key,
model_name=model_name
backend\pipeline\segmenters\trajectory_hybrid.py:3:tracknet_hybrid and wasb_hybrid were copy-
paste twins ? CODE_MAP carried a

```

```

backend\pipeline\segmenters\trajectory_hybrid.py:30:from backend.pipeline.detectors.base import
DetectorRunner
backend\pipeline\segmenters\trajectory_hybrid.py:62:        """Return (runner, detector-specific
detection_params extras) for the default (WSL) path."""
backend\pipeline\segmenters\trajectory_hybrid.py:68:        """Return (runner, extras) for the
Linux-native (no-WSL) path. Only families with a
backend\pipeline\segmenters\trajectory_hybrid.py:73:        f"Linux-native runner. Use
detector_impl='auto' (WSL) or 'stub'."
backend\pipeline\segmenters\trajectory_hybrid.py:82:        ``"stub"`` swaps in a deterministic
CPU runner (no GPU/WSL) so the whole pipeline
backend\pipeline\segmenters\trajectory_hybrid.py:84:        in the Linux-native WASB runner (no
WSL/conda-activate; GPU box) via
backend\pipeline\segmenters\trajectory_hybrid.py:86:        (the WSL runner) so the default
production path is unchanged
backend\pipeline\segmenters\trajectory_hybrid.py:96:        from
backend.pipeline.detectors.stub_runner import (
backend\pipeline\segmenters\trajectory_hybrid.py:102:        "%s: detector_impl=stub ?
CPU mock runner (no GPU/WSL).",
backend\pipeline\segmenters\trajectory_hybrid.py:108:        if impl in ("native",
"native_wasb", "wasb_native"):
backend\pipeline\segmenters\trajectory_hybrid.py:110:        "%s: detector_impl=native ?
Linux-native runner (no WSL).",
backend\pipeline\segmenters\trajectory_hybrid.py:129:        One canonical implementation ? the
twins had drifted (wasb relied on
backend\pipeline\segmenters\trajectory_hybrid.py:140:        # --- Fusion seams (identity by default
? wasb_hybrid/tracknet_hybrid unchanged) -----
backend\pipeline\segmenters\trajectory_hybrid.py:211:        api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\trajectory_hybrid.py:212:        api_key =
os.environ.get(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:213:        if not api_key:
backend\pipeline\segmenters\trajectory_hybrid.py:214:            from
backend.pipeline.segmenters._keyhint import api_key_hint
backend\pipeline\segmenters\trajectory_hybrid.py:217:            f"{api_key_env_var}
environment variable is not set! "
backend\pipeline\segmenters\trajectory_hybrid.py:219:            +
api_key_hint(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:228:            provider_name,
api_key=api_key, model_name=model_name
backend\pipeline\segmenters\trajectory_hybrid.py:245:        # indexing.detector_impl="stub") so
the pipeline runs GPU/WSL-free; otherwise
backend\pipeline\segmenters\trajectory_hybrid.py:246:        # it delegates to the subclass
_build_runner (the real WSL/native runner).
backend\pipeline\segmenters\trajectory_hybrid.py:250:        # e.g. detector_impl="native"
for a family without a native runner (tracknet, or
backend\pipeline\segmenters\trajectory_hybrid.py:251:        # fusion with a tracknet
substrate). An unavailable detector is a failure, not a

```

35. assistant (2026-07-09T02:41:16.600Z)

Now let me examine the AI-handoff behavior of the hybrid segmenters (do they require a cloud key?), the GPU usage in detectors, and the config/compute-placement models.

36. user (2026-07-09T02:41:18.606Z)

```

185         # so it is NOT confused with a genuine 0-rally video. A LEGITIMATE empty match
(a trajectory
186         # was produced but no candidate/AI-confirmed rallies) still returns a bare
``[]``. Every
187         # caller (worker.cmd_infer, main.py, orchestration) branches on
isinstance(result, Failure).
188         label = self.display_label
189         # --- Sport profile + AI provider wiring (identical across segmenters) ---
190         sport_name = self.config.sport
191         try:

```

```

192         sport_profile = sport_registry.get(sport_name)
193     except KeyError as e:
194         logger.error(str(e))
195         # Unrunnable config, not a 0-rally match (mirrors the gemini segmenter).
196         return Failure(str(e))
197
198     idx_cfg = self.config.indexing
199     skip_ai = bool(idx_cfg.skip_ai_handoff)
200     provider_name = self.config.ai_provider
201     if skip_ai:
202         model_name = "local-noai"
203         logger.info(
204             "%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
205             "be emitted as rallies; no %s handoff, no API key needed.",
206             label,
207             provider_name,
208         )
209     else:
210         model_name = self.config.ai_model
211         api_key_env_var = f"{provider_name.upper()}_API_KEY"
212         api_key = os.environ.get(api_key_env_var)
213         if not api_key:
214             from backend.pipeline.segmenters._keyhint import api_key_hint
215
216             msg = (
217                 f"{api_key_env_var} environment variable is not set! "
218                 f"To run the {provider_name} handoff, set your API key. "
219                 + api_key_hint(api_key_env_var)
220             )
221             logger.error(msg)
222             # A misconfig (unrunnable), NOT a 0-rally match ? surface as Failure so
223             # marked failed instead of a silent empty "success" (mirrors gemini; the
224             # own 0-segment diagnostic names this missing-key case as a top cause).
225             return Failure(msg, is_recoverable=True)
226         try:
227             provider = provider_registry.get(
228                 provider_name, api_key=api_key, model_name=model_name
229             )
230         except KeyError as e:
231             logger.error(str(e))
232             return Failure(str(e))
233
234     video_duration = get_video_duration(video_path)
235     if video_duration <= 0:
236         logger.error("Invalid video duration.")
237         # An unreadable/broken video, not a 0-rally match.
238         return Failure(
239             message=f"{label}: invalid/unreadable video duration
240             ({video_duration}).",
241             fps, frame_width = self._probe_fps_width(video_path)
242
243     # --- Runner config + windowing thresholds ---
244     # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
245     # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
246     # it delegates to the subclass _build_runner (the real WSL/native runner).
247     try:
248         runner, runner_params = self._resolve_runner(idx_cfg)
249     except NotImplementedError as e:
250         # e.g. detector_impl="native" for a family without a native runner
251         # fusion with a tracknet substrate). An unavailable detector is a failure,
252         not a

```

```

252             # 0-rally match ? refuse cleanly as a Failure instead of crashing or a false
success.
253             logger.error("%s: %s", label, e)
254             return Failure(message=f"{label}: {e}")
255
256         velocity_thresh = getattr(idx_cfg, self.family).velocity_threshold
257         merge_gap = idx_cfg.dead_time_merge_gap
258         min_window_duration = idx_cfg.min_action_window_duration
259         # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.

```

37. user (2026-07-09T02:41:20.110Z)

```

70         or unknown provider) that ``process_video`` propagates ? so an unrunnable config
is marked
71         FAILED with a reason instead of a silent 0-rally "success" (the wf294 loud-
failures bar;
72         mirrors the gemini/trajectory segmenters). A genuine "ran, found nothing" is
unaffected.
73         """
74         # Get sport profile config
75         sport_name = self.config.sport
76         try:
77             sport_registry.get(sport_name)
78         except KeyError as e:
79             logger.error(str(e))
80             return Failure(str(e))
81
82         # Get AI provider and model config
83         provider_name = self.config.ai_provider
84         model_name = self.config.ai_model
85
86         # Get appropriate API key based on the provider
87         api_key_env_var = f"{provider_name.upper()}_API_KEY"
88         api_key = os.environ.get(api_key_env_var)
89         if not api_key:
90             from backend.pipeline.segmenters._keyhint import api_key_hint
91
92             msg = (
93                 f"{api_key_env_var} environment variable is not set! "
94                 f"To run the {provider_name} segmenter, set your API key. "
95                 + api_key_hint(api_key_env_var)
96             )
97             logger.error(msg)
98             return Failure(msg, is_recoverable=True)
99
100         try:
101             provider = provider_registry.get(
102                 provider_name, api_key=api_key, model_name=model_name
103             )
104         except KeyError as e:
105             logger.error(str(e))
106             return Failure(str(e))
107
108         logger.info(
109             f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}"
110         )
111         return provider
112
113     def _collect_candidate_windows(
114         self,
115         video_id: str,
116         video_path: str,
117         video_duration: float,

```

```

118         idx_cfg: Dict[str, Any],
119         chunk_duration: float,
120         chunk_overlap: float,
121         velocity_thresh: float,
122         merge_gap: float,
123         frame_skip: int,
124         chunks_dir: str,
125     ) -> List[Dict[str, Any]]:
126         """Phase 2: chunk the video, extract per-chunk action windows (pipelined or
127         sequential), then dedup-merge overlapping windows across chunks. Verbatim
128         move."""
129         step = chunk_duration - chunk_overlap
130         chunk_starts = []

```

38. user (2026-07-09T02:41:21.728Z)

24:the person cue is HEAVY-only and never instantiated under Gemini/motion/cpu-mock), and tests can

```

59:     ``config`` (reads ``indexing.{use_gpu, detector_model, detector_score_threshold,
60:         self.device: Optional[str] = None
61:         import torch
62:         use_gpu = idx_cfg.use_gpu
63:         self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"
64:         if use_gpu and not torch.cuda.is_available():
65:             "use_gpu is True but CUDA is not available. Falling back to CPU."
66:         logger.info(f"Loading torchvision detector '{model_name}' on {self.device}...")
67:         self.model.to(self.device)
68:         import torch
69:         torch.from_numpy(rgb).permute(2, 0, 1).float().div(255.0).to(self.device)
70:         boxes = out["boxes"].detach().cpu().numpy()
71:         labels = out["labels"].detach().cpu().numpy()
72:         scores = out["scores"].detach().cpu().numpy()

```

39. assistant (2026-07-09T02:41:35.627Z)

Now let me map the config models (all the tunable knobs), compute placement, confirm motion is CPU-only, and read the eval/regression harnesses.

40. user (2026-07-09T02:41:39.860Z)

```

18:class ValidationRelevanceConfig(BaseModel):
19:    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
20:
21:class ValidationConfig(BaseModel):
22:    min_blur_threshold: float = 20.0
23:    "recordings, and other encoders may use different GOP sizes ? tune "
24:
25:class TrackNetConfig(BaseModel):
26:
27:class WasbIndexConfig(BaseModel):
28:    # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
29:
30:class FusionConfig(BaseModel):
31:
32:class IndexingConfig(BaseModel):
33:    default_segmenter: str = "yolo_hybrid"
34:    use_gpu: bool = True
35:    detector_impl: str = "auto"
36:    skip_ai_handoff: bool = False
37:    @field_validator("default_segmenter")
38:
39:class OutputConfig(BaseModel):
40:
41:class WatermarkConfig(BaseModel):
42:    (under `stitching.watermark` in config.json) to turn it off. The text is fully
43:
44:class SdRenditionConfig(BaseModel):
45:
46:class StitchingRenditionsConfig(BaseModel):
47:
48:class StitchingConfig(BaseModel):
49:    begin_padding: float = 0.5
50:    end_padding: float = 0.5
51:    min_shot_count: int = 1

```

```

382: # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on the
388: # Video encoder for the per-clip trim re-encode (the branding watermark forces a re-
encode; the
390: # ``hevc_nvenc`` are the L4 HARDWARE encoders ? dramatically faster on the 4K re-encode
that
391: # dominated the 2026-07-08 CUJ stitch (#521). The compiler probes the requested encoder
ONCE and
393: # cloud compile-dispatch forwards ``deployment.cloud_stitch_encoder`` onto this for the
L4 run;
395: encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "libx264"
397: # watermark drawn ONCE ? a single encode ? instead of the per-clip re-encode + concat-
copy path
398: # (#521 Part-3). The branding watermark forces a re-encode of every output frame either
way (that
407: single_pass_stitch: bool = True
408: watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)
414:class LoggingConfig(BaseModel):
424:class SecurityConfig(BaseModel):
457:class StorageConfig(BaseModel):
498:class PhasePlacementConfig(BaseModel):
503:     ``backend/config/placement.py``) or an explicit ``"control_plane"`` /
``"cloud_run_l4"``.
504:     ``control_plane`` runs the phase in-process on the CPU-only control-plane;
``cloud_run_l4``
506:     Resolve via ``backend.config.resolve_phase_placement(cfg, phase)`` ? never read these
directly.""
508:     validation: Optional[Literal["control_plane", "cloud_run_l4"]] = None
509:     segment: Optional[Literal["control_plane", "cloud_run_l4"]] = None
510:     compile: Optional[Literal["control_plane", "cloud_run_l4"]] = None
513:class DeploymentConfig(BaseModel):
522:     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the loader.
530:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
532:     # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound Cloud
Run Job
534:     # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
544:     # control-plane, segment on the L4 iff compute_target==cloud_run). Set a phase to
relocate it by
545:     # config alone. Resolve via ``backend.config.resolve_phase_placement`` (never read the
sub-fields
546:     # directly ? the resolver folds in the legacy compute_target default).
547:     phase_placement: PhasePlacementConfig = Field(default_factory=PhasePlacementConfig)
548:     # Video encoder the cloud compile-dispatch forwards onto ``stitching.encoder`` for the
L4 stitch
549:     # (#521). ``h264_nvenc`` uses the L4's hardware encoder for the 4K re-encode the
watermark forces;
551:     # the control-plane's own local stitch always uses ``stitching.encoder`` (libx264 by
default).
552:     cloud_stitch_encoder: Literal["libx264", "h264_nvenc", "hevc_nvenc"] = "h264_nvenc"
567:     # is only launched for the segment(detect) phase. So validation=cloud_run_l4 with
segment on
570:     if self.phase_placement.validation == "cloud_run_l4":
571:         seg = self.phase_placement.segment or (
572:             "cloud_run_l4" if self.compute_target == "cloud_run" else "control_plane"
574:         if seg != "cloud_run_l4":
576:             "deployment.phase_placement.validation='cloud_run_l4' requires segment
to also "
578:             "phase_placement.segment='cloud_run_l4' (or compute_target='cloud_run'),
or keep "
579:             "validation on the control_plane."
584:class BillingConfig(BaseModel):
605:class FlywheelConfig(BaseModel):
627:class AppConfig(BaseModel):

```

```

1      """Phase ? machine placement resolution (#521).
2
3      A *phase placement* answers one question per pipeline phase ? **where does the heavy
compute for
4      this phase run?** ? with a config knob (`deployment.phase_placement`) instead of code.
The three
5      placeable phases and their values:
6
7          validation ? control_plane | cloud_run_l4
8          segment    ? control_plane | cloud_run_l4    (a.k.a. "detect")
9          compile     ? control_plane | cloud_run_l4    (a.k.a. "stitch")
10
11      ``control_plane`` runs the phase IN-PROCESS on the (CPU-only) control-plane service;
``cloud_run_l4``
12      dispatches it to the GPU/NVENC Cloud Run worker Job (the same L4 detection already
uses). This module
13      is the SINGLE source of truth so ``backend.orchestration`` (segment + compile) and the
worker never
14      re-derive the mapping.
15
16      **Default = today's behaviour (pure / additive).** When a phase's knob is unset
(``None``) the
17      resolver reproduces the pre-#521 placement:
18
19          validation ? control_plane                                (the authoritative
gate, #512/#513)
20          segment    ? cloud_run_l4 IFF deployment.compute_target == "cloud_run", else
control_plane
21          compile     ? control_plane                                (the in-process
ffmpeg stitch)
22
23      so a config with no ``phase_placement`` block behaves byte-identically to before this
module existed.
24      """
25
26      from __future__ import annotations
27
28      from typing import Any
29
30      # Phase identities (kept as constants so callers don't stringly-type the phase names).
31      PHASE_VALIDATION = "validation"
32      PHASE_SEGMENT = "segment"
33      PHASE_COMPILE = "compile"
34      PHASES: tuple[str, ...] = (PHASE_VALIDATION, PHASE_SEGMENT, PHASE_COMPILE)
35
36      # Placement targets.
37      PLACE_CONTROL_PLANE = "control_plane"
38      PLACE_CLOUD_L4 = "cloud_run_l4"
39      PLACEMENTS: tuple[str, ...] = (PLACE_CONTROL_PLANE, PLACE_CLOUD_L4)
40
41
42      def resolve_phase_placement(cfg: Any, phase: str) -> str:
43          """Where does ``phase`` run ? ``"control_plane"`` or ``"cloud_run_l4"``?
44
45          Precedence: an explicit ``deployment.phase_placement.<phase>`` wins; otherwise the
legacy default
46          (mirrors the pre-#521 behaviour, see the module docstring). Never raises ? an absent
47          ``deployment`` / ``phase_placement`` degrades to the legacy default.
48          """
49          deployment = getattr(cfg, "deployment", None)
50          placement = getattr(deployment, "phase_placement", None)
51          explicit = getattr(placement, phase, None) if placement is not None else None
52          if explicit:
53              return str(explicit)
54          # Legacy defaults ? reproduce today's placement exactly.

```



```

55         if phase == PHASE_SEGMENT:
56             compute_target = (getattr(deployment, "compute_target", "") or "").strip()
57             return PLACE_CLOUD_L4 if compute_target == "cloud_run" else PLACE_CONTROL_PLANE
58         # validation + compile default to the control-plane (their pre-#521 home).
59         return PLACE_CONTROL_PLANE
60

```

42. user (2026-07-09T02:41:41.262Z)

```

1     import logging
2     import time
3     from typing import Any, Dict, List, Optional, Tuple
4
5     import cv2
6     import numpy as np
7
8     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
9     from backend.results import Failure, Result, Success # standardized contract
10
11     logger = logging.getLogger(__name__)
12
13
14     class MotionPlaySegmenter(BasePlaySegmenter):
15         @property
16         def name(self) -> str:
17             return "motion"
18
19         @property
20         def description(self) -> str:
21             return "Traditional CV pixel-motion heuristics tracking frame differences."
22
23         def process_video(
24             self,
25             video_id: str,
26             video_path: str,
27             player_pool: Optional[List[str]] = None,
28             max_frames: Optional[int] = None,
29         ) -> "Result[List[Dict[str, Any]]]":
30             """
31             Processes a video to detect play segments using motion heuristics and
32             populates SQLite with detected segments. The ``player_pool`` arg is
33             accepted for ABC signature compatibility but no longer used (the motion
34             segmenter has no ground truth for server/receiver ? see _populate_db).
35             """
36             cap = cv2.VideoCapture(video_path)
37             if not cap.isOpened():
38                 logger.error(f"Segmenter could not open video: {video_path}")
39                 return Failure(
40                     message=f"Could not open video: {video_path}", is_recoverable=False

```

43. assistant (2026-07-09T02:41:51.631Z)

Now let me read the eval/regression harnesses and inventory the scripts/tools/nightly directories for Q6.

44. user (2026-07-09T02:41:53.148Z)

```

1     """CLI to evaluate detected rallies against hand-annotated ground truth.
2
3     Scores predictions already stored in the SQLite DB (run the pipeline first) ? no
4     API calls, fully deterministic. See docs/EVALUATION.md for the workflow.
5
6     Examples
7     -----

```

```

8      Create an empty annotation file to fill in:
9          python run_eval.py --scaffold --annotations rallies.csv
10
11      Score a video's detected rallies (identify it by file or by stored video_id):
12          python run_eval.py --video "C:/path/match.mp4" --annotations rallies.csv
13          python run_eval.py --video-id <md5> --annotations rallies.csv --stage both --iou 0.5
14
15      Show which rallies were missed / spurious, and dump a JSON report:
16          python run_eval.py --video-id <md5> --annotations rallies.csv --show-failures --json
report.json
17      """
18
19      import argparse
20      import json
21      import logging
22      import os
23      import sys
24
25      from backend.config import default_db_path
26      from backend.eval.annotations import load_annotations, write_scaffold
27      from backend.eval.harness import STAGES, evaluate, format_report_text, report_to_dict
28      from backend.infrastructure.database import Database
29      from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
30
31      _EPILOG = """\
32      Prerequisite:
33          The harness scores predictions read from the SQLite DB ? it does NOT run the
34          pipeline. So you must first PROCESS the video (e.g. via run_process_and_stitch.py)
35          so its detections land in output/sports_indexer.db. Then point this tool at the
36          same video file (--video) or its stored MD5 (--video-id). If you get
37          "database not found" or zero predictions, the video hasn't been processed yet.
38
39      See docs/EVALUATION.md for the annotation format and how to read the output.
40      """
41
42
43      def build_parser() -> argparse.ArgumentParser:
44          p = argparse.ArgumentParser(
45              description="Evaluate detected rallies vs ground-truth annotations.",
46              epilog=_EPILOG,
47              formatter_class=argparse.RawDescriptionHelpFormatter,
48          )
49          p.add_argument(
50              "--annotations", help="Path to the ground-truth rally CSV (start,end per row)."
51          )
52          p.add_argument(
53              "--video", help="Path to the video file (its MD5 becomes the video_id)."
54          )
55          p.add_argument(
56              "--video-id", help="Stored video_id (MD5) to evaluate, instead of --video."
57          )
58          p.add_argument(
59              "--db", default=None, help="SQLite DB path (default: from config.json)."
60          )
61          p.add_argument(
62              "--stage",
63              choices=("candidates", "final", "both"),
64              default="both",
65              help="Which prediction stage(s) to score (default: both).",
66          )
67          p.add_argument(
68              "--detector",
69              default=None,
70              help="Only score candidates from this detector (e.g. yolo_hybrid,
tracknet_hybrid).",

```

```

71     )
72     p.add_argument(
73         "--iou", type=float, default=0.5, help="IoU match threshold (default: 0.5).")
74     )
75     p.add_argument(
76         "--pr-curve",
77         action="store_true",
78         help="Include an IoU sweep (0.1..0.9) in the report.",
79     )
80     p.add_argument(
81         "--show-failures",
82         action="store_true",
83         help="List missed (FN) and spurious (FP) rallies with timestamps.",
84     )
85     p.add_argument(
86         "--json",
87         dest="json_out",
88         default=None,
89         help="Write a JSON report to this path.",
90     )
91     p.add_argument(
92         "--scaffold",
93         action="store_true",
94         help="Write an empty annotation CSV to --annotations and exit.",
95     )
96     return p
97
98
99 def main(argv=None) -> int:
100     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
101     args = build_parser().parse_args(argv)
102
103     # Scaffold mode: just write the template and exit.
104     if args.scaffold:
105         if not args.annotations:
106             print("ERROR: --scaffold requires --annotations <path>", file=sys.stderr)
107             return 2
108         write_scaffold(args.annotations)
109         print(f"Wrote empty annotation file: {args.annotations}")
110         return 0
111
112     if not args.annotations:
113         print("ERROR: --annotations is required (or use --scaffold).", file=sys.stderr)
114         return 2
115
116     # Resolve the video_id.
117     if args.video_id:
118         video_id = args.video_id
119     elif args.video:
120         if not os.path.exists(args.video):
121             print(f"ERROR: video not found: {args.video}", file=sys.stderr)
122             return 2
123         print("Computing video MD5...")
124         video_id = get_file_md5(args.video)
125         print(f"video_id = {video_id}")
126     else:
127         print("ERROR: provide --video <path> or --video-id <md5>.", file=sys.stderr)
128         return 2
129
130     db_path = args.db or default_db_path()
131     if not os.path.exists(db_path):
132         print(
133             f"ERROR: database not found at {db_path}. Run the pipeline first.",
134             file=sys.stderr,
135         )

```

```

136         return 2
137
138     gt = load_annotations(args.annotations)
139     if not gt:
140         print(
141             f"WARNING: no ground-truth rallies in {args.annotations}.", file=sys.stderr
142         )
143
144     stages = list(STAGES) if args.stage == "both" else [args.stage]
145     db = Database(db_path)
146     report = evaluate(
147         db,
148         video_id,
149         gt,
150         stages,
151         iou_threshold=args.iou,
152         detector=args.detector,
153         with_pr_curve=args.pr_curve,
154     )
155
156     print(format_report_text(report, show_failures=args.show_failures))
157
158     if args.json_out:
159         with open(args.json_out, "w") as f:
160             json.dump(report_to_dict(report), f, indent=2)
161             print(f"\nJSON report written to {args.json_out}")
162
163     return 0
164
165
166 if __name__ == "__main__":
167     raise SystemExit(main())
168

```

45. user (2026-07-09T02:41:53.646Z)

```

1     """CLI: regression-test the production tool against golden-set ground truth.
2
3     Obtains ground truth from an annotation provider (a tier), scores the video's
4     stored predictions, and compares to a snapshotted baseline. Exit code 1 on a
5     detected regression so it can gate CI. See docs/GOLDEN_SET_IMPLEMENTATION.md.
6
7     Examples
8     -----
9     Snapshot the current numbers as the baseline (first run / after a real improvement):
10     python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
--update-baseline
11
12     Check for a regression (exit 1 if precision/recall dropped beyond tolerance):
13     python run_regression.py --video-id <md5> --tier file --annotations rallies.csv
14     """
15
16     import argparse
17     import json
18     import logging
19     import os
20     import sys
21
22     from backend.annotations import annotation_registry
23     from backend.config import default_db_path
24     from backend.eval import regression
25     from backend.infrastructure.database import Database
26
27
28     def build_parser() -> argparse.ArgumentParser:

```

```

29     p = argparse.ArgumentParser(
30         description="Regression-test the tool vs golden-set ground truth."
31     )
32     p.add_argument(
33         "--video-id",
34         required=True,
35         help="Stored video_id (MD5) whose predictions to score.",
36     )
37     p.add_argument(
38         "--tier",
39         default="file",
40         help=f"Annotation provider tier. Available:
{annotation_registry.list_available()}",
41     )
42     p.add_argument(
43         "--annotations",
44         help="video_ref for the provider (e.g. CSV path for the 'file' tier).",
45     )
46     p.add_argument("--stage", choices=("candidates", "final"), default="final")
47     p.add_argument(
48         "--detector", default=None, help="Restrict candidate scoring to one detector."
49     )
50     p.add_argument("--iou", type=float, default=0.5)
51     p.add_argument("--tolerance", type=float, default=regression.DEFAULT_TOLERANCE)
52     p.add_argument("--db", default=None)
53     p.add_argument(
54         "--baseline",
55         default=regression.DEFAULT_BASELINE_PATH,
56         help="Baseline JSON path.",
57     )
58     p.add_argument(
59         "--update-baseline",
60         action="store_true",
61         help="Snapshot the current numbers as the new baseline instead of checking.",
62     )
63     p.add_argument(
64         "--allow-missing-baseline",
65         action="store_true",
66         help="Treat a missing baseline as a pass (exit 0) instead of a gate error (exit
3).",
67     )
68     p.add_argument("--json", dest="json_out", default=None)
69     return p
70
71
72 def main(argv=None) -> int:
73     logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
74     args = build_parser().parse_args(argv)
75
76     db_path = args.db or default_db_path()
77     if not os.path.exists(db_path):
78         print(
79             f"ERROR: database not found at {db_path}. Run the pipeline first.",
80             file=sys.stderr,
81         )
82     return 2
83
84     db = Database(db_path)
85
86     result = regression.regress_with_provider(
87         db,
88         video_id=args.video_id,
89         video_ref=args.annotations,
90         tier=args.tier,
91         stage=args.stage,
92         iou_threshold=args.iou,

```

```

92         detector=args.detector,
93         tolerance=args.tolerance,
94         baseline_path=args.baseline,
95     )
96
97     print(regression.format_result(result))
98
99     if args.update_baseline:
100         regression.save_baseline(
101             result.video_id,
102             result.stage,
103             result.precision,
104             result.recall,
105             result.f1,
106             path=args.baseline,
107             iou_threshold=result.iou_threshold,
108             detector=args.detector,
109             gt_fingerprint=result.gt_hash,
110         )
111         print(f"Baseline updated in {args.baseline}.")
112
113     if args.json_out:
114         with open(args.json_out, "w") as f:
115             json.dump(result.as_dict(), f, indent=2)
116             print(f"JSON written to {args.json_out}")
117
118     # Exit codes: 1 = regression, 3 = no baseline to compare (gate cannot pass), 0 = ok.
119     if result.regressed:
120         return 1
121     if (
122         not result.has_baseline
123         and not args.update_baseline
124         and not args.allow_missing_baseline
125     ):
126         print(
127             "ERROR: no baseline to compare against ? the gate cannot verify anything. "
128             "Snapshot one with --update-baseline (then commit it), or pass "
129             "--allow-missing-baseline to accept this run.",
130             file=sys.stderr,
131         )
132         return 3
133     return 0
134
135
136 if __name__ == "__main__":
137     raise SystemExit(main())
138

```

46. user (2026-07-09T02:41:54.004Z)

```

1  #!/usr/bin/env python
2  """Run the FULL test suite for the badminton-highlight-indexer.
3
4  A single, discoverable entry point so you (or a reviewer / CI) can verify the
5  whole suite before opening a PR:
6
7      python run_all_tests.py          # full suite, quiet
8      python run_all_tests.py -v       # verbose (passthrough to pytest)
9      python run_all_tests.py -k config # filter by keyword (passthrough)
10     python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
11
12     Anything after the script name is passed straight through to pytest.
13
14     Notes
15     ----

```

```

16 - The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
17 - The observability **reporting** tests skip cleanly if the optional ``obsreport``
18 package isn't installed (they use ``pytest.importorskip``). To exercise them too:
19     pip install -e ../sports-obsreport --no-build-isolation
20 - Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
21 """
22
23 from __future__ import annotations
24
25 import importlib.util
26 import subprocess
27 import sys
28
29
30 def _installed(module: str) -> bool:
31     try:
32         return importlib.util.find_spec(module) is not None
33     except (ImportError, ValueError):
34         return False
35
36
37 def main() -> int:
38     args = list(sys.argv[1:])
39     # Default to quiet unless the caller chose a verbosity.
40     if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
41         args = ["-q", *args]
42     cmd = [sys.executable, "-m", "pytest", *args]
43
44     obs = (
45         "available"
46         if _installed("obsreport")
47         else "NOT installed ? reporting tests will SKIP"
48     )
49     bar = "=" * 72
50     print(bar)
51     print(" Full test suite: " + " ".join(cmd))
52     print(f" obsreport (optional reporting dep): {obs}")
53     print(bar, flush=True)
54
55     rc = subprocess.run(cmd).returncode
56
57     print(bar)
58     print(
59         " RESULT: "
60         + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
61     )
62     print(bar)
63     return rc
64
65
66 if __name__ == "__main__":
67     raise SystemExit(main())
68

```

47. assistant (2026-07-09T02:42:04.291Z)

The hashing module and run_eval reference a `scripts/run\_process\_and\_stitch.py` ? highly relevant to Q2. Let me inventory scripts/, tools/, and output/nightly_regression/, and find that script.

48. user (2026-07-09T02:42:06.380Z)

scripts\run_process_and_stitch.py

49. user (2026-07-09T02:42:07.594Z)

=== scripts/ ===

total 180

drwxr-xr-x	1	avidu	197609	0	Jul	5	19:39	.
drwxr-xr-x	1	avidu	197609	0	Jul	8	21:45	..
drwxr-xr-x	1	avidu	197609	0	Jul	2	20:19	__pycache__
-rwxr-xr-x	1	avidu	197609	4968	Jul	5	19:39	bundle_ui.sh
-rw-r--r--	1	avidu	197609	8349	Jul	2	20:16	doctor.py
-rwxr-xr-x	1	avidu	197609	10386	Jun	29	13:47	install.py
-rw-r--r--	1	avidu	197609	5150	Jun	29	13:47	nightly_email.py
-rw-r--r--	1	avidu	197609	31914	Jun	29	13:47	nightly_reelcount.py
-rw-r--r--	1	avidu	197609	3606	Jun	25	16:24	nightly_regression.ps1
-rw-r--r--	1	avidu	197609	32807	Jun	29	13:47	nightly_regression.py
-rw-r--r--	1	avidu	197609	1391	Jun	15	14:11	README.md
-rw-r--r--	1	avidu	197609	3466	Jun	25	16:24	register_nightly_task.ps1
-rw-r--r--	1	avidu	197609	9151	Jun	29	13:47	run_phase3_eval.py
-rw-r--r--	1	avidu	197609	8389	Jun	29	13:47	run_process_and_stitch.py
-rwxr-xr-x	1	avidu	197609	7744	Jun	29	13:47	uninstall.py
-rwxr-xr-x	1	avidu	197609	6088	Jun	29	13:47	verify_compute_path.py

=== tools/ ===

total 84

drwxr-xr-x	1	avidu	197609	0	Jul	2	19:37	.
drwxr-xr-x	1	avidu	197609	0	Jul	8	21:45	..
-rw-r--r--	1	avidu	197609	0	Jun	13	21:02	__init__.py
drwxr-xr-x	1	avidu	197609	0	Jul	2	19:42	__pycache__
-rw-r--r--	1	avidu	197609	6318	Jul	2	08:13	check_md_links.py
-rw-r--r--	1	avidu	197609	20281	Jun	29	13:47	cleanup_caches.py
-rw-r--r--	1	avidu	197609	18607	Jun	29	13:47	generate_highlights.py
-rw-r--r--	1	avidu	197609	7175	Jun	29	13:47	leak_scan.py
-rw-r--r--	1	avidu	197609	6740	Jul	2	19:37	upload_retention.py

=== output/ ===

total 5049699

drwxr-xr-x	1	avidu	197609	0	Jul	8	16:00	.
drwxr-xr-x	1	avidu	197609	0	Jul	8	21:45	..
-rw-r--r--	1	avidu	197609	6054	Jun	19	13:05	_phase1_regress.log
-rw-r--r--	1	avidu	197609	1301	Jun	19	13:04	_phase1_suite.log
-rw-r--r--	1	avidu	197609	12990	Jun	19	13:10	_phase1_suite_fixed.log
-rw-r--r--	1	avidu	197609	41236	Jun	19	21:27	_staleness_fixes.json
-rw-r--r--	1	avidu	197609	13127	Jun	19	21:42	_suite_courtgate.log
-rw-r--r--	1	avidu	197609	2613	Jun	19	22:00	_verify_bxh2.pkl
drwxr-xr-x	1	avidu	197609	0	Jun	19	13:17	_work19
-rw-r--r--	1	avidu	197609	241664	Jun	19	17:27	19june.db
-rw-r--r--	1	avidu	197609	31432	Jun	19	16:02	19june_extract.log
-rw-r--r--	1	avidu	197609	22924	Jun	19	17:28	19june_extract_rerun.log
-rw-r--r--	1	avidu	197609	3032	Jun	19	18:48	19june_local_vs_gemini.md
drwxr-xr-x	1	avidu	197609	0	Jun	20	00:17	19june_work
-rw-r--r--	1	avidu	197609	42226	Jul	4	04:48	a6_r2_ablation.json
-rw-r--r--	1	avidu	197609	0	Jul	4	01:49	a7_mahadevpura_1_default.err.log
-rw-r--r--	1	avidu	197609	2273	Jul	4	01:58	a7_mahadevpura_1_default.out.log
-rw-r--r--	1	avidu	197609	0	Jul	4	01:59	a7_remaining_default.err.log
-rw-r--r--	1	avidu	197609	44754	Jul	4	04:42	a7_remaining_default.out.log
drwxr-xr-x	1	avidu	197609	0	Jul	4	01:45	a7_smoke
-rw-r--r--	1	avidu	197609	0	Jul	4	04:44	a8_audio_augment.err.log
-rw-r--r--	1	avidu	197609	1885	Jul	4	04:46	a8_audio_augment.out.log
-rw-r--r--	1	avidu	197609	2253	Jun	14	16:07	ab_experiment_n6.json
-rw-r--r--	1	avidu	197609	9016	Jun	8	18:24	adarsh_avi_full_vs_human.csv
-rw-r--r--	1	avidu	197609	131699	Jul	4	04:44	adarsh_avi_singles_golden_features.json
-rw-r--r--	1	avidu	197609	561	Jun	9	18:14	adarsh_avi_vs_human.csv
-rw-r--r--	1	avidu	197609	1721	Jun	6	21:23	adarsh_baseline_gated.csv
-rw-r--r--	1	avidu	197609	2023	Jun	6	21:15	adarsh_baseline_segments.csv
-rw-r--r--	1	avidu	197609	1628	Jun	6	22:40	adarsh_gemini_fullvideo.csv
-rw-r--r--	1	avidu	197609	1670	Jun	6	22:21	adarsh_gemini_refined.csv

-rw-r--r--	1	avidu	197609	269	Jun	6	22:03	adarsh_gemini_smoke.csv
-rw-r--r--	1	avidu	197609	1327	Jun	6	21:29	adarsh_min5s_segments.csv
-rw-r--r--	1	avidu	197609	2895	Jun	6	20:54	adarsh_tuned_segments.csv
-rw-r--r--	1	avidu	197609	1970	Jun	17	13:39	Badminton_BXH_2_rallies.csv
-rw-r--r--	1	avidu	197609	24330	Jul	4	04:45	Badminton_BXH_2_golden_features.json
-rw-r--r--	1	avidu	197609	36864	May	19	23:07	badminton_indexer.db
-rw-r--r--	1	avidu	197609	1915	Jun	18	11:49	Boxhill_Doubles_rallies.csv
-rw-r--r--	1	avidu	197609	47264	Jul	4	04:46	Boxhill_Doubles_golden_features.json
drwxr-xr-x	1	avidu	197609	0	Jul	8	15:58	chunks
-rw-r--r--	1	avidu	197609	3162	Jun	22	15:12	cloud_lovo_manifest.json
-rw-r--r--	1	avidu	197609	3979353	Jun	11	20:43	e2e_reel.mp4
-rw-r--r--	1	avidu	197609	2481	Jun	11	20:43	e2e_server_log.txt
-rw-r--r--	1	avidu	197609	67688	Jun	14	13:35	fusion_golden.log
drwxr-xr-x	1	avidu	197609	0	Jun	14	16:06	gateA_served_verify
-rw-r--r--	1	avidu	197609	62555	Jul	4	04:45	gbaaddy_golden_features.json
-rw-r--r--	1	avidu	197609	5645	Jun	8	21:56	gbaaddy_vs_human.csv
drwxr-xr-x	1	avidu	197609	0	Jun	17	14:43	gem_newclips
-rw-r--r--	1	avidu	197609	90112	Jun	19	18:36	gemini_19june.db
-rw-r--r--	1	avidu	197609	147964	Jun	19	14:19	gemini_19june.log
-rw-r--r--	1	avidu	197609	7708	Jun	19	18:47	gemini_19june_rest.log
-rw-r--r--	1	avidu	197609	372	Jun	19	14:01	gemini_GX010142_19june.json
-rw-r--r--	1	avidu	197609	307	Jun	19	14:08	gemini_GX010143_19june.json
-rw-r--r--	1	avidu	197609	347	Jun	19	14:19	gemini_GX010145_19june.json
-rw-r--r--	1	avidu	197609	77824	Jun	18	13:47	gemini_long.db
-rw-r--r--	1	avidu	197609	288	Jun	18	14:02	gemini_long_run.log
-rw-r--r--	1	avidu	197609	1207870728	Jun	13	15:03	GX010125_highlights.mp4
-rw-r--r--	1	avidu	197609	2083468872	Jun	13	20:19	GX010125_highlights_min5.mp4
-rw-r--r--	1	avidu	197609	6756	Jun	13	20:19	gx010125_min5_stitch.log
drwxr-xr-x	1	avidu	197609	0	Jun	14	10:48	GX010125_run
-rw-r--r--	1	avidu	197609	108744	Jun	13	15:03	gx010125_run.log
-rw-r--r--	1	avidu	197609	35165	Jun	12	22:29	gx010125_transcode.log
-rw-r--r--	1	avidu	197609	2286	Jun	16	22:27	GX010128_rallies.csv
-rw-r--r--	1	avidu	197609	41639	Jul	4	04:45	GX010128_golden_features.json
-rw-r--r--	1	avidu	197609	1489	Jun	18	10:13	GX010137_rallies.csv
-rw-r--r--	1	avidu	197609	43554	Jul	4	04:45	GX010137_golden_features.json
-rw-r--r--	1	avidu	197609	1274	Jun	18	06:49	GX010141_rallies.csv
-rw-r--r--	1	avidu	197609	20787	Jul	4	04:45	GX010141_golden_features.json
-rw-r--r--	1	avidu	197609	1715	Jun	18	12:16	GX020094_rallies.csv
-rw-r--r--	1	avidu	197609	76605	Jul	4	04:46	GX020094_golden_features.json
-rw-r--r--	1	avidu	197609	1800	Jun	17	13:15	GX030094_rallies.csv
-rw-r--r--	1	avidu	197609	62024	Jul	4	04:45	GX030094_golden_features.json
drwxr-xr-x	1	avidu	197609	0	Jun	15	23:42	highlights_work
drwxr-xr-x	1	avidu	197609	0	Jun	17	14:43	highlights_work_gem
-rw-r--r--	1	avidu	197609	4503	Jun	18	12:26	human_lovo_manifest.json
-rw-r--r--	1	avidu	197609	2727	Jun	17	13:56	human_lovo_manifest.json.bak
-rw-r--r--	1	avidu	197609	4997	Jul	3	22:49	human_lovo_manifest.local.json
-rw-r--r--	1	avidu	197609	160570312	Jun	10	16:46	kushagra_1280.mp4
-rw-r--r--	1	avidu	197609	2659	Jun	10	17:01	kushagra_1280.report.json
-rw-r--r--	1	avidu	197609	1166	Jun	10	17:01	kushagra_1280.report.md
-rw-r--r--	1	avidu	197609	480	Jun	10	17:01	kushagra_1280.summary.md
drwxr-xr-x	1	avidu	197609	0	Jun	16	01:40	kushagra_highlights_work
-rw-r--r--	1	avidu	197609	16195	Jul	4	04:44	kushagra_singles_golden_features.json
-rw-r--r--	1	avidu	197609	3200	Jun	8	18:42	kushagra_vs_human.csv
-rw-r--r--	1	avidu	197609	1496443316	Jun	13	22:30	KushagraAviHighlight-Trailor.mp4
-rw-r--r--	1	avidu	197609	64537	Jul	4	04:45	largetest_doubles_golden_features.json
-rw-r--r--	1	avidu	197609	6120	Jun	8	21:03	largetestvideo_vs_human.csv
-rw-r--r--	1	avidu	197609	370	Jun	6	22:40	local_calib_manifest.json
-rw-r--r--	1	avidu	197609	86016	Jun	16	02:27	local_compare.db
-rw-r--r--	1	avidu	197609	662	Jun	6	22:51	local_rally_model.json
-rw-r--r--	1	avidu	197609	9205	Jun	16	20:39	local_vs_gemini_6.json
-rw-r--r--	1	avidu	197609	1873	Jun	16	20:39	local_vs_gemini_6.md
-rw-r--r--	1	avidu	197609	5719	Jun	16	01:57	local_vs_gemini_report.md
-rw-r--r--	1	avidu	197609	465	Jun	17	12:49	mahadevpura_1_rallies.csv
-rw-r--r--	1	avidu	197609	1345	Jul	4	04:45	mahadevpura_1_golden_features.json
-rw-r--r--	1	avidu	197609	1719	Jun	16	19:30	mahadevpura_2_rallies.csv

```

-rw-r--r-- 1 avidu 197609      12764 Jul  4 04:45 mahadevpura_2_golden_features.json
-rw-r--r-- 1 avidu 197609       1451 Jun 10 18:07 mahadevpura_gemini_refined.csv
-rw-r--r-- 1 avidu 197609     43410 Jul  4 04:45 mahadevpura_singles_golden_features.json
-rw-r--r-- 1 avidu 197609    90194321 Jun 11 11:21 mahadevpura_smoke_180s.mp4
-rw-r--r-- 1 avidu 197609     3214 Jun 11 18:53 mahadevpura_smoke_180s.report.json
-rw-r--r-- 1 avidu 197609     1565 Jun 11 18:53 mahadevpura_smoke_180s.report.md
-rw-r--r-- 1 avidu 197609      486 Jun 11 18:53 mahadevpura_smoke_180s.summary.md
-rw-r--r-- 1 avidu 197609     3438 Jun  9 10:40 mahadevpura_vs_human.csv
-rw-r--r-- 1 avidu 197609 125200877 Jun 10 16:31 MahadevpuraSingles_highlights.mp4
-rw-r--r-- 1 avidu 197609     8977 Jun 19 22:05 multicourt_analysis.md
drwxr-xr-x 1 avidu 197609        0 Jun 23 03:00 nightly_regression
-rw-r--r-- 1 avidu 197609      305 Jun 19 21:52 person_extract.log
-rw-r--r-- 1 avidu 197609   259889 Jun 19 21:52 person_GX010142.json
-rw-r--r-- 1 avidu 197609     1934 May 26 04:04 phase3_probe.json
-rw-r--r-- 1 avidu 197609   114688 May 26 03:53 phase3_yolo.db
-rw-r--r-- 1 avidu 197609   141594 Jun 10 16:21 ramp_probe.mp4
-rw-r--r-- 1 avidu 197609    16248 Jun 16 20:52 REAL_FOOTAGE_VALIDATION_REPORT.md
drwxr-xr-x 1 avidu 197609        0 Jul  8 15:58 reports
-rw-r--r-- 1 avidu 197609     9707 Jul  4 04:42 run_telemetry.jsonl
-rw-r--r-- 1 avidu 197609    17587 Jul  8 12:10 sd_reel_full_tests.log
drwxr-xr-x 1 avidu 197609        0 Jun 19 13:30 smoke19
-rw-r--r-- 1 avidu 197609    47104 Jun 11 18:53 smoke4_log.txt
-rw-r--r-- 1 avidu 197609   258048 Jun 17 14:41 sports_indexer.db
-rw-r--r-- 1 avidu 197609    24576 May 20 12:04 test_indexer.db
-rw-r--r-- 1 avidu 197609     350 Jun  6 22:30 testlarge_gemini_fullvideo.csv
-rw-r--r-- 1 avidu 197609     270 Jun  6 22:24 testlarge_gemini_rallies.csv
-rw-r--r-- 1 avidu 197609    10505 Jul  4 04:45 testlarge_short_golden_features.json
-rw-r--r-- 1 avidu 197609     1171 Jun  8 21:56 testlarge_vs_human.csv
-rw-r--r-- 1 avidu 197609    22395 Jun 13 22:28 trim70.log
drwxr-xr-x 1 avidu 197609        0 Jun 11 20:43 uploads
drwxr-xr-x 1 avidu 197609        0 Jun 20 15:25 wasb
drwxr-xr-x 1 avidu 197609        0 Jun 16 00:04 wasb_sxs
-rw-r--r-- 1 avidu 197609    1526 Jun  6 20:03 wasb_tuned_segments.csv
-rw-r--r-- 1 avidu 197609    1185 Jun  6 20:03 wasb_tuned_segments_baseline.csv
-rw-r--r-- 1 avidu 197609    2997 Jun  6 20:03 wasb_tuned_segments_vs_golden.csv

```

=== output/nightly_regression/ ===

total 229

```

drwxr-xr-x 1 avidu 197609        0 Jun 23 03:00 .
drwxr-xr-x 1 avidu 197609        0 Jul  8 16:00 ..
-rw-r--r-- 1 avidu 197609    9677 Jun 18 03:00 2026-06-18.json
-rw-r--r-- 1 avidu 197609    3753 Jun 18 03:00 2026-06-18.md
-rw-r--r-- 1 avidu 197609   18354 Jun 19 13:05 2026-06-19.json
-rw-r--r-- 1 avidu 197609    5856 Jun 19 13:05 2026-06-19.md
-rw-r--r-- 1 avidu 197609   18353 Jun 20 03:00 2026-06-20.json
-rw-r--r-- 1 avidu 197609    5493 Jun 20 03:00 2026-06-20.md
-rw-r--r-- 1 avidu 197609   18353 Jun 21 03:00 2026-06-21.json
-rw-r--r-- 1 avidu 197609    5493 Jun 21 03:00 2026-06-21.md
-rw-r--r-- 1 avidu 197609   18353 Jun 22 03:00 2026-06-22.json
-rw-r--r-- 1 avidu 197609    5493 Jun 22 03:00 2026-06-22.md
-rw-r--r-- 1 avidu 197609   18353 Jun 23 03:00 2026-06-23.json
-rw-r--r-- 1 avidu 197609    5493 Jun 23 03:00 2026-06-23.md
-rw-r--r-- 1 avidu 197609    163 Jun 29 12:37 LATEST_STATUS.txt
drwxr-xr-x 1 avidu 197609        0 Jun 29 12:37 logs

```

50. assistant (2026-07-09T02:42:14.963Z)

The `scripts/run\_process\_and\_stitch.py` is the key process+stitch script for Q2. Let me read it, plus check the nightly harness and scripts for timing instrumentation.

51. user (2026-07-09T02:42:16.477Z)

```

1      import json
2      import logging
3      import os

```

```

4
5     from dotenv import load_dotenv
6
7     load_dotenv()
8
9     # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
10    # Quiet third-party HTTP chatter by default; re-honoured against the config knob
11    # (logging.verbose_http) once the config is loaded in main().
12    from backend.utils.logging_setup import setup_logging
13
14    setup_logging()
15
16    from backend.config import load_config
17    from backend.infrastructure.database import Database
18    from backend.pipeline.segmenters import segmenter_registry
19    from backend.pipeline.stitcher.compiler import VideoCompiler
20    from backend.results import unwrap_or_failure
21    from backend.tools.export import format_rally_summary
22    from backend.utils.hashing import compute_video_id
23
24    logger = logging.getLogger(__name__)
25
26
27    def main():
28        import argparse
29
30        parser = argparse.ArgumentParser(
31            description="Dev runner: process a video -> index rallies -> stitch a highlight
reel."
32        )
33        parser.add_argument(
34            "--video-path", required=True, help="Path to the input video file."
35        )
36        parser.add_argument(
37            "--output-dir",
38            default=None,
39            help="Where to write the stitched reel (default: config output.output_dir).",
40        )
41        parser.add_argument(
42            "--output-filename", default="highlights.mp4", help="Stitched reel filename."
43        )
44        args = parser.parse_args()
45        video_path = os.path.abspath(args.video_path)
46        output_filename = args.output_filename
47
48        print("Loading config...")
49        cfg = load_config() # typed AppConfig ? single source of defaults
50        # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
quiet).
51        setup_logging(verbose_http=cfg.logging.verbose_http)
52        output_dir = args.output_dir or cfg.output.output_dir # no hardcoded OS path
53
54        print("Verifying files exist...")
55        if not os.path.exists(video_path):
56            print(f"Error: Video file not found at {video_path}")
57            return
58
59        os.makedirs(cfg.output.output_dir, exist_ok=True)
60        db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
61        db = Database(db_path)
62
63        # Calculate MD5 of the video to uniquely identify it
64        import time
65
66        print(

```

```

67         "Calculating video MD5 (this might take a few seconds on large files)...",
68         flush=True,
69     )
70     t0 = time.time()
71     video_id = compute_video_id(video_path)
72     print(f"Calculated MD5: {video_id} (took {time.time() - t0:.1f}s)", flush=True)
73
74     segments = []
75     cached_video = db.get_video(video_id)
76     cache_hit = False
77
78     if cached_video and cached_video.get("status") == "processed":
79         segments = db.query_play_segments(video_id, {})
80         if len(segments) > 0:
81             print(
82                 f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
with {len(segments)} parsed play segments.",
83                 flush=True,
84             )
85             choice = input(
86                 "Would you like to (1) Run stitching on the cached segments, or (2)
Reprocess the video with Gemini from scratch? [1/2]: "
87             ).strip()
88             if choice == "1":
89                 print(
90                     "[CACHE] Using cached play segments. Skipping segmenter API call.",
91                     flush=True,
92                 )
93                 cache_hit = True
94             else:
95                 print("[CACHE] Ignoring cache...", flush=True)
96         else:
97             print(
98                 "[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.",
99                 flush=True,
100             )
101
102     if not cache_hit:
103         print("[CACHE] Initializing clean indexing...", flush=True)
104         db.clear_video_data(video_id)
105
106         print(
107             "Bypassing OpenCV validations for performance on massive files...",
108             flush=True,
109         )
110         db.add_video(video_id, video_path, "processed", [])
111
112         print("Running segmenter & indexing...", flush=True)
113         seg_name = cfg.indexing.default_segmenter
114         segmenter_cls = segmenter_registry.get(seg_name)
115         if not segmenter_cls:
116             print(f"[FAIL] Segmenter '{seg_name}' not found.")
117             return
118         print(f"Using segmenter: {seg_name}", flush=True)
119         segmenter = segmenter_cls(db, cfg)
120         # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
121         # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
122         # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
123         segments, failure = unwrap_or_failure(
124             segmenter.process_video(video_id, video_path)
125         )

```

```

126         if failure is not None:
127             print(f"[FAIL] Segmentation failed: {failure.message}")
128             return
129         print(f"Successfully indexed {len(segments)} play segments.", flush=True)
130
131     if not segments:
132         print("No play segments detected. Cannot compile highlights.")
133         return
134
135     end_padding = cfg.stitching.end_padding
136     begin_padding = cfg.stitching.begin_padding
137     min_shot_count = cfg.stitching.min_shot_count
138
139     if cache_hit:
140         print("\n--- Stitching Customizations ---")
141         try:
142             bpad_input = input(
143                 f"Enter begin_padding in seconds (default {begin_padding}): "
144             ).strip()
145             if bpad_input:
146                 begin_padding = float(bpad_input)
147
148             pad_input = input(
149                 f"Enter end_padding in seconds (default {end_padding}): "
150             ).strip()
151             if pad_input:
152                 end_padding = float(pad_input)
153
154             shot_input = input(
155                 f"Enter min_shot_count to filter by (default {min_shot_count}): "
156             ).strip()
157             if shot_input:
158                 min_shot_count = int(shot_input)
159         except ValueError:
160             print(
161                 "[WARNING] Invalid input detected, falling back to defaults from
config.json."
162             )
163
164     print(
165         f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...",
166         flush=True,
167     )
168
169     # Filter segments where shot_count >= min_shot_count
170     time_pairs = []
171     skipped_count = 0
172     for s in segments:
173         meta = s.get("metadata", {})
174         if isinstance(meta, str):
175             try:
176                 meta = json.loads(meta)
177             except Exception:
178                 meta = {}
179         shot_count = meta.get("shot_count")
180         # B1/P9: shot_count is None when the segmenter has no ground truth (motion
181         # segmenter) ? keep such segments rather than crashing on None >= int.
182         if shot_count is None or shot_count >= min_shot_count:
183             time_pairs.append((s["start_time"], s["end_time"]))
184         else:
185             skipped_count += 1
186
187     if skipped_count > 0:
188         print(

```

```

189         f"[FILTER] Excluded {skipped_count} play segments with fewer than
{min_shot_count} shots.",
190         flush=True,
191     )
192     print(
193         f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.", flush=True
194     )
195
196     # End-of-run rally table for manual QA ? sourced from the produced play_segments so
197     # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
198     # aligned rows) so it survives even if the table is the last thing the owner scans.
199     def _log_rally_summary() -> None:
200         logger.info("\n%s", format_rally_summary(segments))
201
202     if not time_pairs:
203         print(
204             "No play segments passed the filtering criteria. Cannot compile highlights."
205         )
206         _log_rally_summary()
207         return
208
209     compiler = VideoCompiler(
210         output_dir,
211         end_padding=end_padding,
212         begin_padding=begin_padding,
213         watermark=config.stitching.watermark,
214     )
215     try:
216         out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
217         print(f"Stitched highlight video saved to: {out_path}", flush=True)
218     except Exception as e:
219         print(f"Error during stitching: {e}", flush=True)
220
221     _log_rally_summary()
222
223
224     if __name__ == "__main__":
225         main()
226

```

52. user (2026-07-09T02:42:18.765Z)

```

scripts\nightly_regression.ps1:7:      (CPU, seconds ? the WASB trajectories are CACHED) and diffs
vs the frozen baseline. Writes a
scripts\nightly_email.py:47: def resolve_secret(env: Optional[dict] = None) -> Optional[str]:
scripts\nightly_email.py:61:         return
client.access_secret_version(name=secret_ref).payload.data.decode(
scripts\nightly_email.py:98:     password = resolve_secret(env)
scripts\nightly_regression.py:4: seconds ? the WASB GPU trajectories are CACHED, only the
windowing/scorer changes) and diffs
scripts\nightly_regression.py:168:         max_window_duration=6.0,
scripts\nightly_regression.py:181:         max_window_duration=0.0,
scripts\run_process_and_stitch.py:67:     "Calculating video MD5 (this might take a few
seconds on large files)...",
scripts\run_process_and_stitch.py:70:     t0 = time.time()
scripts\run_process_and_stitch.py:72:     print(f"Calculated MD5: {video_id} (took {time.time() -
t0:.1f}s)", flush=True)
scripts\run_process_and_stitch.py:143:         f"Enter begin_padding in seconds (default
{begin_padding}): "
scripts\run_process_and_stitch.py:149:         f"Enter end_padding in seconds (default
{end_padding}): "
scripts\verify_compute_path.py:2: ""Verify the compute path an /api/process job ACTUALLY took
(COMPUTE_DECOUPLED validation method).
scripts\verify_compute_path.py:7: This closes the loop: the picker asked for a path, and we prove
the engine took it (no silent fallback).

```

```

scripts\verify_compute_path.py:51:         description="Prove which compute path an /api/process
job took."
scripts\verify_compute_path.py:99:         started = time.monotonic()
scripts\verify_compute_path.py:102:         if time.monotonic() - started > args.timeout_sec:
scripts\verify_compute_path.py:116:         print(f"FAIL: intended path {args.expect!r} but the
job took {actual!r}")
scripts\verify_compute_path.py:118:         print(f"OK: the job took the intended compute path:
{actual}")
scripts\install.py:238:
timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
scripts\run_phase3_eval.py:193:         t0 = time.time()
scripts\run_phase3_eval.py:206:         print(f"          -> {len(segs)} segments in
{time.time() - t0:.0f}s")
scripts\nightly_reelcount.py:16:  ? COST of the run ? VM uptime (or processing wall) × machine
$/hr via ``backend.billing`` ? USD + INR.
scripts\nightly_reelcount.py:77:         billed_seconds: float,
scripts\nightly_reelcount.py:80:         gpu_seconds: float = 0.0,
scripts\nightly_reelcount.py:82:         """Run cost from billed wall-seconds × machine rate ? USD +
INR (``backend.billing`` arithmetic).
scripts\nightly_reelcount.py:84:         ``billed_seconds`` is the VM uptime (boot?delete) when the
orchestrator passes ``--vm-uptime-seconds``;
scripts\nightly_reelcount.py:85:         otherwise it's the in-process pipeline wall time (a lower
bound ? the VM also pays for boot/install).
scripts\nightly_reelcount.py:89:         rate_per_sec = machine_usd_per_hr / 3600.0
scripts\nightly_reelcount.py:90:         usage = {billing.WALL_SECONDS: float(billed_seconds)}
scripts\nightly_reelcount.py:91:         rates = {billing.WALL_SECONDS: rate_per_sec}
scripts\nightly_reelcount.py:94:         "billed_seconds": round(float(billed_seconds), 1),
scripts\nightly_reelcount.py:95:         "gpu_seconds": round(float(gpu_seconds), 1),
scripts\nightly_reelcount.py:111:         Each run result: ``{name, reel_count, ok, error, quality:
{tol_f1,...}|None, wall_seconds}``.
scripts\nightly_reelcount.py:276:         f"billed {cost.get('billed_seconds', 0):.0f}s @
${cost.get('machine_usd_per_hr', 0)}/hr",
scripts\nightly_reelcount.py:416:         "wall_seconds": 0.0,
scripts\nightly_reelcount.py:433:         t0 = time.monotonic()
scripts\nightly_reelcount.py:435:         wall = time.monotonic() - t0
scripts\nightly_reelcount.py:437:         return None, None, wall, getattr(result, "message",
"segmenter failed")
scripts\nightly_reelcount.py:445:         wall,
scripts\nightly_reelcount.py:448:         return len(segs), _segments_to_intervals(segs), wall,
None
scripts\nightly_reelcount.py:452:         total_wall = 0.0
scripts\nightly_reelcount.py:467:         "wall_seconds": 0.0,
scripts\nightly_reelcount.py:470:         count, intervals, wall, err = _process(local,
os.path.join(td, "run1.db"))
scripts\nightly_reelcount.py:471:         total_wall += wall
scripts\nightly_reelcount.py:479:         "wall_seconds": round(total_wall, 2),
scripts\nightly_reelcount.py:487:         count2, intervals2, wall2, err2 = _process(
total_wall += wall2
scripts\nightly_reelcount.py:490:         "wall_seconds": round(total_wall, 2),
scripts\nightly_reelcount.py:504:         "wall_seconds": round(total_wall, 2),
scripts\nightly_reelcount.py:529:

```

[Showing results with pagination = limit: 50]

53. assistant (2026-07-09T02:42:37.342Z)

This is rich. `nightly\_reelcount.py` is the performance/cost harness with `wall\_seconds`. Let me read its core, the nightly_regression flow, and the compiler's stitch signature.

54. user (2026-07-09T02:42:39.434Z)

```

1         """Nightly END-TO-END engine regression ? reel-count + quality + cost (the GCP-VM
nightly).
2
3         Runs the **configured + checkpointed** pipeline (the real WASB GPU detector, as a
deployment serves

```

```

4      it) end-to-end on 2-3 short owned golden videos and asserts the **number of reels does
not change**
5      vs a frozen baseline. Unlike ``scripts/nightly_regression.py`` (PR #202 ? a CPU re-score
of CACHED
6      trajectories, the windowing/scorer only), this one runs the *full* pipeline on the *real
model*, so it
7      needs a GPU ? hence it runs on an ephemeral GCP GPU VM
(``deploy/gcp/nightly_regression/``), emails the
8      owner the report, and self-deletes. It is **default-additive**: a tripwire, not the
promotion gate.
9
10     What it asserts / reports, per the owner's ask ("quality metrics + regression report +
cost"):
11     ? REEL COUNT per video ? ``len(play_segments)`` after segmentation; the detected
highlight clips.
12     Pure-CV / WASB detection is deterministic for a fixed (video, config, weights), so
this is asserted
13     **EXACT** (with a re-run-once guard for the one known flake: a transient DB-insert-
None, see $guard).
14     ? QUALITY METRICS per video (when golden labels are provided) ? the R2 tolerance-match
block from
15     ``backend.eval.rally_seg_eval`` (tol_f1 / f1@0.5 / f1@0.25), compared with a small
tolerance.
16     ? COST of the run ? VM uptime (or processing wall) × machine $/hr via
``backend.billing`` ? USD + INR.
17
18     Extensible pattern (add a video = one manifest row + re-freeze):
19     ``eval_baselines/reelcount_manifest.json`` ? the videos (gs:// / local / gdrive://
URIs) + segmenter
20     ``eval_baselines/reelcount_baseline.json`` ? the frozen expected counts (regenerated,
never hand-edited)
21
22     Usage
23     -----
24     Freeze the baseline from the current configured+checkpointed engine (after an intended
change, or first run):
25     python scripts/nightly_reelcount.py --update-baseline
26
27     Nightly check (exit 1 on regression; writes output/nightly_reelcount/<date>.md; emails
if --email):
28     python scripts/nightly_reelcount.py --email
29
30     Exit codes: 0 pass · 1 regression (reel-count changed / a video errored / quality
dropped) · 2 operational
31     (manifest/videos missing, nothing ran) · 3 no baseline · 4 stale (corpus/segmenter
changed ? re-freeze).
32     """
33
34     from __future__ import annotations
35
36     import argparse
37     import datetime as _dt
38     import json
39     import os
40     import sys
41     from dataclasses import dataclass, field
42     from typing import Any, Dict, List, Optional, Tuple
43
44     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
45     if REPO_ROOT not in sys.path:
46         sys.path.insert(0, REPO_ROOT)
47
48     # billing is pure (typing only) ? safe to import at module top.
49     from backend import billing # noqa: E402
50

```



```

51     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_manifest.json")
52     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "reelcount_baseline.json")
53     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_reelcount")
54
55     #: Quality metrics we report + (when labels exist) gate ? higher is better, small
56     tolerance.
57     QUALITY_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
58     DEFAULT_QUALITY_TOL = (
59         0.02 # quality is float/labeled ? a looser tripwire than the exact reel-count
60     )
61     DEFAULT_FX_INR_PER_USD = 83.0
62     #: Fallback machine price if the manifest doesn't pin one (T4 on n1-standard-8, asia-
63     south1, ~mid-2026).
64     DEFAULT_MACHINE_USD_PER_HR = 0.35
65
66     @dataclass(frozen=True)
67     class Tolerances:
68         quality_tol: float = DEFAULT_QUALITY_TOL
69
70         def to_dict(self) -> Dict[str, Any]:
71             return {"quality_tol": self.quality_tol}
72
73     # ----- #
74     # Pure cost math (wraps backend.billing ? no IO).
75     # ----- #
76     def cost_report(
77         billed_seconds: float,
78         machine_usd_per_hr: float,
79         fx_inr_per_usd: float,
80         gpu_seconds: float = 0.0,
81     ) -> Dict[str, Any]:
82         """Run cost from billed wall-seconds × machine rate ? USD + INR (``backend.billing``
83         arithmetic).
84         ``billed_seconds`` is the VM uptime (boot?delete) when the orchestrator passes
85         ``--vm-uptime-seconds``;
86         otherwise it's the in-process pipeline wall time (a lower bound ? the VM also pays
87         for boot/install).
88         This is a **best-effort estimate** (uptime × rate), not the authoritative GCP bill ?
89         for drift,
90         reconcile against the Cloud Billing export. A non-positive rate is clamped to 0 (?
91         $0, never negative)."""
92         machine_usd_per_hr = max(0.0, float(machine_usd_per_hr))
93         rate_per_sec = machine_usd_per_hr / 3600.0
94         usage = {billing.WALL_SECONDS: float(billed_seconds)}
95         rates = {billing.WALL_SECONDS: rate_per_sec}
96         usd, breakdown = billing.compute_cost_usd(usage, rates)
97         return {
98             "billed_seconds": round(float(billed_seconds), 1),
99             "gpu_seconds": round(float(gpu_seconds), 1),
100             "machine_usd_per_hr": float(machine_usd_per_hr),
101             "usd": usd,
102             "inr": billing.to_inr(usd, fx_inr_per_usd),
103             "breakdown": breakdown,
104         }
105
106     # ----- #
107     # Pure summary + comparison core (no IO ? unit-testable with plain dicts).
108     # ----- #
109     def summarize_results(
110         run_results: List[Dict[str, Any]], segmenter: str, cost: Dict[str, Any]
111     ) -> Dict[str, Any]:

```

```

109         """Compact, comparable snapshot from per-video run results.
110
111         Each run result: ``{name, reel_count, ok, error, quality: {tol_f1,...}|None,
wall_seconds}``.
112         A video that errored keeps ``reel_count=None`` + ``ok=False`` (so the diff flags it,
never hides it)."""
113         per_video: Dict[str, Any] = {}
114         for r in run_results:
115             per_video[r["name"]] = {
116                 "reel_count": r.get("reel_count"),
117                 "ok": bool(r.get("ok", False)),
118                 "error": r.get("error"),
119                 "quality": r.get("quality"),
120             }
121         return {
122             "segmenter": segmenter,
123             "cost": cost,
124             "per_video": per_video,
125             "n": len(per_video),
126         }
127
128
129     @dataclass
130     class Finding:

```

55. user (2026-07-09T02:42:40.335Z)

```

400
401     from backend.eval import rally_seg_eval
402     from backend.infrastructure.database import Database
403     from backend.pipeline.segmenters import segmenter_registry
404     from backend.results import Failure
405     from backend.utils.hashing import compute_video_id
406
407     name = video["name"]
408     seg_cls = segmenter_registry.get(segmenter)
409     if not seg_cls:
410         return {
411             "name": name,
412             "reel_count": None,
413             "ok": False,
414             "error": f"segmenter '{segmenter}' not registered",
415             "quality": None,
416             "wall_seconds": 0.0,
417         }
418
419     def _process(
420         local: str, db_path: str
421     ) -> Tuple[
422         Optional[int], Optional[List[Tuple[float, float]]], float, Optional[str]
423     ]:
424         random.seed(
425             0
426         ) # close motion.py's unseeded metadata RNG so secondary fields are
reproducible too
427         db = Database(db_path)
428         video_id = compute_video_id(local)
429         # Register the video row FIRST (the production pipeline does this before
segmenting,
430         # backend/main.py::_execute_processing) ? play_segments has an FK to videos.id,
so a
431         # missing parent row makes every add_play_segment fail and the reel count come
back 0.
432         db.add_video(video_id, local, "processed", [])
433         t0 = time.monotonic()

```

```

434         result = seg_cls(db, config).process_video(video_id, local)
435         wall = time.monotonic() - t0
436         if isinstance(result, Failure):
437             return None, None, wall, getattr(result, "message", "segmenter failed")
438         segs = result.value if hasattr(result, "value") else result
439         if not isinstance(
440             segs, list
441         ): # contract guard ? report a violation clearly, don't crash
442             return (
443                 None,
444                 None,
445                 wall,
446                 f"segmenter returned {type(segs).__name__}, expected a list of
segments",
447             )
448         return len(segs), _segments_to_intervals(segs), wall, None
449
450     import tempfile
451
452     total_wall = 0.0
453     try:
454         with tempfile.TemporaryDirectory(prefix="nightly_reel_") as td:
455             # fetch INTO the workdir ? gs:// / s3:// / gdrive:// require a dst (no
passthrough). A bad
456             # clip's fetch error is reported as THIS clip's error (compare() ?
regression), never raised
457             # out to sink the whole run.
458             try:
459                 local = _fetch_to(video["uri"], td)
460             except Exception as e: # noqa: BLE001
461                 return {
462                     "name": name,
463                     "reel_count": None,
464                     "ok": False,
465                     "error": f"fetch failed: {e}",
466                     "quality": None,
467                     "wall_seconds": 0.0,
468                 }
469             gts = _load_gts(video.get("labels_uri"), td)
470             count, intervals, wall, err = _process(local, os.path.join(td, "run1.db"))
471             total_wall += wall
472             if err is not None:
473                 return {
474                     "name": name,
475                     "reel_count": None,
476                     "ok": False,
477                     "error": err,
478                     "quality": None,
479                     "wall_seconds": round(total_wall, 2),
480                 }
481             # re-run-once guard for the transient DB-drop flake
482             if (
483                 rerun_on_mismatch
484                 and baseline_count is not None
485                 and count != baseline_count
486             ):
487                 count2, intervals2, wall2, err2 = _process(
488                     local, os.path.join(td, "run2.db")
489                 )
490                 total_wall += wall2
491                 if err2 is None and count2 is not None:
492                     count, intervals = (
493                         count2,
494                         intervals2,
495                     ) # the reproduced (or corrected) value

```

```

496         else:
497             # The re-run itself errored ? the count is INCONCLUSIVE, not a
confirmed regression.
498             return {
499                 "name": name,
500                 "reel_count": None,
501                 "ok": False,
502                 "error": f"count {count}?baseline {baseline_count} on run 1, but
the re-run failed: {err2}",
503                 "quality": None,
504                 "wall_seconds": round(total_wall, 2),
505             }
506
507         quality = None
508         if gts and intervals is not None:
509             row = rally_seg_eval.score_intervals(intervals, gts)
510             quality = {
511                 k: row[k]
512                 for k in (
513                     "tol_f1",
514                     "f1@0.5",
515                     "f1@0.25",
516                     "tol_precision",
517                     "tol_recall",
518                 )
519                 if k in row
520             }
521         except Exception as e: # noqa: BLE001 ? one clip's failure (decode/OOM/etc.) must
NOT sink the whole
522             # nightly: report THIS clip as an error (compare() ? regression names it); other
clips still run.
523             return {
524                 "name": name,
525                 "reel_count": None,
526                 "ok": False,
527                 "error": f"pipeline raised: {e}",
528                 "quality": None,
529                 "wall_seconds": round(total_wall, 2),
530             }
531
532         return {
533             "name": name,
534             "reel_count": count,
535             "ok": True,
536             "error": None,
537             "quality": quality,
538             "wall_seconds": round(total_wall, 2),
539         }

```

56. assistant (2026-07-09T02:42:55.425Z)

Let me confirm the compiler's stitch signature, the validators registry, and verify whether the live pipeline (orchestration/main) has any per-phase timing.

57. user (2026-07-09T02:42:58.542Z)

```

40: def _probe_encoder_available(encoder: str) -> bool:
82:     def __init__(
89:         single_pass: bool = False,
97:         self.watermark = self._normalize_watermark(watermark)
107:         # StitchingConfig.single_pass_stitch=True, which orchestration passes explicitly.
See
108:         # _compile_single_pass.
109:         self.single_pass = bool(single_pass)
282:         wm = self.watermark

```

```

609:     def _compile_single_pass(
762:     def compile_highlights(
823:         if self.single_pass:
824:             sp_result = self._compile_single_pass(

```

58. user (2026-07-09T02:42:59.432Z)

```

1     from backend.pipeline.validators.base import ValidationResult, VideoValidator, registry
2     from backend.pipeline.validators.blur import BlurValidator
3     from backend.pipeline.validators.format import FormatValidator
4     from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
5     from backend.pipeline.validators.relevance import RelevanceValidator
6     from backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
7     from backend.pipeline.validators.scene_cut import SceneCutDensityValidator
8
9     # Register all default validators in sequence (dependencies first)
10    registry.register(FormatValidator())
11    registry.register(ResolutionFPSValidator())
12    registry.register(PTSContinuityValidator())
13    registry.register(SceneCutDensityValidator())
14    registry.register(BlurValidator())
15    registry.register(RelevanceValidator())
16
17    __all__ = ["registry", "VideoValidator", "ValidationResult"]
18

```

59. user (2026-07-09T02:43:01.051Z)

No matches found

60. assistant (2026-07-09T02:43:09.907Z)

Let me read the exact `compile\_highlights` signature and the validators' `run\_all\_with\_context` to complete the call-site citations.

61. user (2026-07-09T02:43:11.398Z)

```

762     def compile_highlights(
763         self,
764         raw_video_path: str,
765         segments: List[Tuple[float, float]],
766         output_filename: str,
767     ) -> str:
768         """
769         Extracts selected segments from a raw video and stitches them together
770         into a seamless highlights file, applying end_padding to prevent abrupt endings.
771
772         .. warning::
773             This method uses a two-stage codec contract: per-clip re-encodes must use
774             IDENTICAL encoder settings (codec, preset, CRF, pixel format) so the final
775             concat can stream-copy (`-c copy`) without re-encoding. Changing the
776             clip encode settings without updating the concat assumptions will produce
777             broken output.
778         """
779         if not segments:
780             raise ValueError("No highlight segments provided to compile.")
781
782         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
783         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
784         original_count = len(segments)
785         segments = self._merge_overlapping(segments)
786         if len(segments) < original_count:
787             logger.info(
788                 "Merged %d overlapping/duplicate segment(s) before stitching: %d ->

```

```

%d.",
789         original_count - len(segments),
790         original_count,
791         len(segments),
792     )
793     if not segments:
794         raise ValueError("No valid (start < end) highlight segments after merge.")
795
796     if not os.path.exists(raw_video_path):
797         raise FileNotFoundError(
798             f"[COMPILER ERROR] Source video file not found: {raw_video_path}"
799         )
800
801     # Defense-in-depth: never let the output name escape output_dir (the API also
802     # validates this at the request boundary).
803     output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
804
805     # Probe video duration so we can detect out-of-range segments
806     video_duration = self._get_video_duration(raw_video_path)
807     if video_duration > 0:
808         logger.info(f"Source video duration: {video_duration:.2f}s")
809     else:
810         logger.warning(
811             "Video duration unknown. Cannot validate segment boundaries against
video length."
812         )
813
814     # We will create temporary clips and stitch them using FFmpeg concat demuxer
815     temp_clips = []
816     skipped_segments = []
817
818     # Create a temp directory within output_dir
819     with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
820         # Fast path (opt-in): stitch the whole reel in ONE ffmpeg pass (concat
filter +
821         # watermark-once). Frame-accurate + ~12-14% faster (#521 Part-3). Returns
None on any
822         # failure so we transparently fall through to the robust per-clip path
below.
823         if self.single_pass:
824             sp_result = self._compile_single_pass(
825                 raw_video_path, segments, output_path, video_duration, tmp_dir
826             )
827             if sp_result is not None:
828                 return sp_result
829             logger.warning(
830                 "[compile] single-pass stitch unavailable/failed; using the per-clip
path."
831             )
832             logger.info(
833                 f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)..."
834             )
835         # Build the (optional) watermark filtergraph once; it is applied during
every
836         # per-clip re-encode so the mark is baked into each clip identically.

```

62. user (2026-07-09T02:43:12.888Z)

```

26: class ValidationResult(BaseModel):
27:     validator_name: str
28:     passed: bool
129:     def run_all_with_context(self, video_path: str, config: Any):
149:         validator_name=val.name,
150:         passed=False,

```

```
183: def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:
```

63. assistant (2026-07-09T02:43:22.777Z)

Let me confirm the key import lines in main.py to cite them precisely.

64. user (2026-07-09T02:43:25.899Z)

```
67:from backend.pipeline.stitcher.compiler import VideoCompiler
70:from backend.reporting import emit_indexer_report
94:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
```

65. user (2026-07-09T02:43:26.614Z)

```
202     class IndexingConfig(BaseModel):
203         default_segmenter: str = "yolo_hybrid"
204         use_gpu: bool = True
205         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
206         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
207         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
208         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
209         # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
210         detector_impl: str = "auto"
211         motion_threshold: float = 0.08
212         min_segment_duration: float = 3.0
213         dead_time_merge_gap: float = 2.0
214         chunk_duration: float = 90.0
215         chunk_overlap: float = 10.0
216         detector_model: str = "fasterrcnn_resnet50_fpn"
217         detector_score_threshold: float = 0.5
218         yolo_velocity_threshold: float = 0.15
219         yolo_frame_skip: int = 3
220         yolo_tracker: str = "bytetrack.yaml"
221         yolo_prefetch_workers: int = 2
222         min_action_window_duration: float = 2.0
223         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
224         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
225         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
226         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
227         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
228         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
229         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
230         max_window_duration: float = 6.0
231         window_min_split_gap: float = (
```

66. assistant (2026-07-09T02:43:34.040Z)

Let me read the WASB native-runner config knobs and the `skip\_ai\_handoff` context to complete the local-GPU config story.

67. user (2026-07-09T02:43:36.010Z)

```
68     class WasbIndexConfig(BaseModel):
69         """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71         Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
```

```

72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     ---
89     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
90     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
91     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
92     device: str = "cuda" # auto | cuda | mps | cpu
93     python_bin: str = (
94         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
95     )
96     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
97     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
98     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
99     keep_frames: bool = False
100    # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the

```

68. user (2026-07-09T02:43:36.506Z)

```

283     ai_chunk_scale_width: int = 1280
284     # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285     # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286     # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287     # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288     # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289     skip_ai_handoff: bool = False
290     # Optional debug knob: trim every video to the first N seconds before processing.
291     debug_duration: Optional[float] = None
292
293     tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294     wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295     fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297     @field_validator("default_segmenter")

```

69. assistant (2026-07-09T02:43:52.621Z)

Let me check whether the WASB/TrackNet detectors can run natively on Windows (in-process) or require WSL ? this matters for the RTX 2070 feasibility story.

70. user (2026-07-09T02:43:54.329Z)


```
total 220
drwxr-xr-x 1 avidu 197609    0 Jul  7 16:57 .
drwxr-xr-x 1 avidu 197609    0 Jun 14 16:28 ..
-rw-r--r-- 1 avidu 197609 1065 Jun 29 13:47 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jul  8 15:58 __pycache__
-rw-r--r-- 1 avidu 197609 1666 Jun 29 13:47 audio_features.py
-rw-r--r-- 1 avidu 197609 11435 Jul  2 08:13 base.py
-rw-r--r-- 1 avidu 197609 5604 Jun 29 13:47 court_gate.py
-rw-r--r-- 1 avidu 197609 2493 Jun 29 13:47 device.py
-rw-r--r-- 1 avidu 197609 10063 Jun 29 13:47 fusion_features.py
-rw-r--r-- 1 avidu 197609 20590 Jul  7 16:57 native_wasb_runner.py
-rw-r--r-- 1 avidu 197609 19923 Jun 29 13:47 person_detector.py
-rw-r--r-- 1 avidu 197609 5447 Jun 29 20:33 rally_gate.py
-rw-r--r-- 1 avidu 197609 8269 Jun 29 13:47 stub_runner.py
-rw-r--r-- 1 avidu 197609 28469 Jul  2 08:13 tracknet_runner.py
-rw-r--r-- 1 avidu 197609 60552 Jul  7 16:57 wasb_infer.py
-rw-r--r-- 1 avidu 197609 10610 Jul  2 08:13 wasb_runner.py
```

71. user (2026-07-09T02:43:54.931Z)

```
1      """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3      The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4      "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5      It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6      Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7      that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8      see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16     * weights / repo / python / scratch come from the environment first
17     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26     Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
27     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28     candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
29     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31     vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
32     artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
```

```

33      (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34      not the numbers.
35      """
36
37      import logging
38      import os
39      import shutil
40      import subprocess
41      from dataclasses import dataclass
42      from typing import List, Optional, Tuple
43
44      from backend.config.models import IndexingConfig
45      from backend.pipeline.detectors.base import DetectorRunner
46      from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48      # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49      from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51      logger = logging.getLogger(__name__)
52
53      # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54      # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55      _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")

```

72. assistant (2026-07-09T02:44:14.857Z)

I have a comprehensive picture. Let me do one final consolidated grep to precisely cite all
`perf\_counter`/`duration\_ms` usages across the repo for Q3.

73. user (2026-07-09T02:44:17.898Z)

```
tests\test_job_worker_module_seam.py:78:     class _FakeTimer:
```

74. assistant (2026-07-09T02:46:59.266Z)

I now have a complete picture. Here is the structured report.

Local End-to-End Pipeline Map ? `badminton-highlight-indexer`

All paths absolute. The headline finding: the `python -m backend.main` CLI single-shot does
`\*\*checksum ? validate ? segment ? report\*\*` but `\*\*never stitches\*\*`; stitching lives on the
API/job path and requires explicit segment IDs. There is `**no aggregated per-phase wall-clock
timing in the live pipeline**` ? the one real timing/cost harness is
`scripts/nightly\_reelcount.py` (segmentation only), and `RunTelemetry` exists but is wired only
into the fusion-golden eval.

Q1 ? End-to-end CLI flow (`python -m backend.main`)

Entry point: `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\main.py:1271`
`def main()` (argparse). The relevant imports:
- `backend\main.py:94` ? `from backend.utils.hashing import compute\_video\_id`
- `backend\main.py:67` ? `from backend.pipeline.stitcher.compiler import VideoCompiler`
- `backend\main.py:70` ? `from backend.reporting import emit\_indexer\_report`
- validators `registry` and `segmenter\_registry` are module-level singletons.

The `\*\*CLI single-shot branch\*\*` is gated at `backend\main.py:1492`:
``python

backend/main.py:1492

```
if args.video_path and not args.server and not args.app:
    ...
```

(a) Checksum / MD5 ? `backend/main.py:1508`

```
```python
```

## backend/main.py:1506-1509

```
local_video = storage.acquire_local_input(args.video_path, storage_cfg)
try:
```

```
 video_id = compute_video_id(local_video)
 was_reingest = db_instance.get_video(video_id) is not None
 ...
```

`compute\_video\_id` is the canonical whole-file MD5, streamed in 64 KB chunks:

`backend\utils\hashing.py:23-33`. **\*\*Not timed\*\*** in `main.py` (no wall-clock print). The only timed MD5 call in the repo is the dev script ? `scripts\run\_process\_and\_stitch.py:70-72` (`t0 = time.time(); ...; took {time.time()-t0:.1f}s`).

### (b) Validation ? `backend/main.py:1513`

```
```python
```

backend/main.py:1512-1519

```
print("[CLI] Validating...")
val_results, val_context = registry.run_all_with_context(
    local_video, global_config
)
failures = [r for r in val_results if not r.passed]
status = "failed" if failures else "processed"
...

```

The registry and its 6 validators (run in dependency order) are defined at

`backend\pipeline\validators\\_\_init\_\_.py:10-15`: `FormatValidator`, `ResolutionFPSValidator`, `PTSContinuityValidator`, `SceneCutDensityValidator`, `BlurValidator`, `RelevanceValidator`.
`run\_all\_with\_context` is `backend\pipeline\validators\base.py:129`. On failure the run halts:
`backend/main.py:1538-1540` (`"[FAIL] Video failed checks. Indexing halted."`). ****Not timed.****

(c) Segmentation / indexing ? `backend/main.py:1543-1555`

Segmenter selection (per-run `--segmenter` flag overrides `indexing.default\_segmenter`, hard fallback `"motion")`:

```
```python
```

## backend/main.py:1524-1526

```
seg_name = args.segmenter or getattr(
 getattr(global_config, "indexing", None), "default_segmenter", "motion"
)
...

```

```
```python
```

backend/main.py:1543-1555

```
segmenter_cls = segmenter_registry.get(seg_name)
if not segmenter_cls:
    print(f"[FAIL] Segmenter '{seg_name}' not found.")
    return
print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
segmenter_actual = seg_name
play_wm, cand_wm = db_instance.segment_watermarks(video_id)
segmenter = segmenter_cls(db_instance, global_config)
result = segmenter.process_video(
    video_id, local_video, max_frames=args.max_frames
)
segmentation_ran = True
...
**Not timed.** Only stage banners are printed.
```

(d) Stitching / compiling ? ****absent from the CLI flow****

The CLI single-shot ends by emitting the per-video observability report in a `finally`, then

```
`return`s ? no stitch:
```

```
```python
```

## **backend/main.py:1568-1593 (abridged)**

```
finally:
```

```
 paths = emit_indexer_report(db=db_instance, video_id=video_id, video_path=local_video, ...)
```

```
 ...
```

```
return
```

```
```
```

Stitching is only reachable via the API/job path (Q2).

****Server/job path (for completeness):**** `backend\orchestration.py:765`

`\_execute\_processing(...)` is the `/api/process` worker equivalent (checksum ? validate ? segment). It also emits the report (`orchestration.py:1093`) and has ****no per-phase timing**** (grep for `perf\_counter`/`monotonic`/`duration\_ms` in `orchestration.py` ? none).

Q2 ? Stitching the FULL reel from the CLI with no user step

No CLI flag exists

`backend.main`'s full argument list is `backend/main.py:1275-1334`: `--video-path, --output-dir, --setup, --debug-clip, --server, --app, --port, --host, --max-frames, --segmenter, --trim-start, --trim-end, --trim-output`. There is ****no `--stitch` / `--compile` / `--make-reel`****. (`--trim-start/--trim-end` at `:1354` stitches a single manual `[start,end]` clip, not the indexed rallies.)

How the reel is compiled today (API ? job ? worker)

`/api/compile` handler `backend/main.py:1097` `\_compile\_highlights(req: CompileRequest, ...)` ? requires the caller to pass explicit `req.segment\_ids` (the SPA sends the user's selected rally IDs):

```
```python
```

## **backend/main.py:1112-1141 (abridged)**

```
video, _kept, out_name = _resolve_compile(
 request.app.state.db, request.app.state.config,
 req.video_id, req.segment_ids, req.output_filename,
)
```

```
...
```

```
request.app.state.db.create_compile_job(job_id, req.video_id, video["filepath"],
 req.segment_ids, out_name, locale=req.locale, owner_id=owner_id, rendition=req.rendition)
...
```

The worker runs `\_execute\_compile` ? `\_stitch\_reel` ? `VideoCompiler.compile\_highlights`:

- `backend\orchestration.py:1559` `\_execute\_compile(cfg, db, job, progress)` ? reads `segment\_ids` from `params` (`orch:1572`), resolves via `\_resolve\_compile`, builds `time\_pairs = [(s["start\_time"], s["end\_time"]) for s in kept]` (`orch:1600`), calls `\_stitch\_reel` (`orch:1601`).
- `backend\orchestration.py:1250` `\_stitch\_reel(...)` ? builds the compiler and stitches (`orch:1294-1322`).
- `backend\pipeline\stitcher\compiler.py:762` `def compile\_highlights(self, raw\_video\_path, segments, output\_filename) -> str`.

`\_resolve\_compile` (`backend\orchestration.py:1147`) is where the `stitching.min\_shot\_count` floor is applied (`orch:1169-1192`).

### **Exact programmatic call to stitch ALL indexed rallies (no SPA)**

**\*\*Option A ? direct compiler call (simplest; this is exactly what the dev script does).\*\*** Query every play segment for the `video\_id`, turn them into time-pairs, and call the compiler:

```
```python
```

pattern copied from scripts/run_process_and_stitch.py:170-216

```
segments = db.query_play_segments(video_id, {}) # ALL rallies for this video
time_pairs = [(s["start_time"], s["end_time"]) for s in segments] # (optionally filter by
min_shot_count)
```

```

compiler = VideoCompiler(
    output_dir,
    begin_padding=cfg.stitching.begin_padding,
    end_padding=cfg.stitching.end_padding,
    watermark=cfg.stitching.watermark,
)
out_path = compiler.compile_highlights(video_path, time_pairs, "highlights.mp4")
...

`db.query_play_segments(video_id, {})` (empty filter = all) is the same call `_resolve_compile`
uses internally (`backend\orchestration.py:1164`).

**Option B ? go through orchestration (applies the min_shot_count floor + watermark
localization):**
```python
segment_ids = [s["id"] for s in db.query_play_segments(video_id, {})]
job = {"video_id": video_id,
 "params": json.dumps({"segment_ids": segment_ids,
 "output_filename": "highlights.mp4",
 "rendition": "original"})}
result = _execute_compile(cfg, db, job) # backend/orchestration.py:1559
...

```

### Ready-made script (nearly turnkey, one caveat)

`scripts\run\_process\_and\_stitch.py` already does **process ? index ? stitch ALL segments** end to end (uses `cfg.indexing.default\_segmenter`, stitches everything passing `min\_shot\_count` at `:169-216`). It is **non-interactive** on a fresh run, BUT it calls `input()` and blocks when the video is already cached-processed (`scripts\run\_process\_and\_stitch.py:85-86` and `:142-158`). For an unattended benchmark, run it against a clean DB (cache miss) or use Option A directly to avoid the prompt entirely.

---

## Q3 ? Existing performance / latency instrumentation

### There IS a telemetry module ? but it is not in the live pipeline

`backend\utils\run\_telemetry.py` ? `class RunTelemetry` (`:135`) is a rotating ~10 KB JSONL "black box" of execution velocity + machine health (GPU temp/util/VRAM via `nvidia-smi`, CPU/RAM via `psutil`):

```

```python

```

backend/utils/run_telemetry.py:186-209 (snapshot derives fps + elapsed_s)

```

now = self._clock()
rec = {"t": self._iso(now), "elapsed_s": round(now - self._t0, 1)}
if phase is not None: rec["phase"] = phase
...
rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
...

```

****Wiring:**** it is instantiated ****only**** in `backend\eval\fusion\_golden.py:322` (label `fusion\_golden`, writes `output/run\_telemetry.jsonl`; snapshots at `:216`, events at `:384-403`). The production CLI (`backend/main.py`) and `orchestration.py` do ****not**** import or use it.

The live pipeline records NO per-phase wall-clock

- Grep for `perf\_counter` / `time.monotonic` / `duration\_ms` / `PhaseTimer` in `backend\orchestration.py` ? ****none****. The only `elapsed` there are minute-granular UI progress strings for cloud-job polling: `\_wait\_progress\_message` (orch:236) and `\_compile\_wait\_message` (orch:1241) ? text only, never persisted.
- Repo-wide, `time.perf\_counter` / `PhaseTimer` / `stopwatch` appear ****nowhere in production**** ? the single hit is a test double `\_FakeTimer` at `tests\test\_job\_worker\_module\_seam.py:78`.

The per-video "observability report" records durations of CONTENT, not of processing

`emit\_indexer\_report` (`backend\reporting\\_\_init\_\_.py:29`) harvests into technical/customer/JSON reports. Its `duration` fields are ****video/segment content durations****, not wall-clock: `backend\reporting\harvest.py:63-65` (meta["duration_s"]), `:111` / `:186` (total_play =

sum(segment durations)`). In-process facts arrive as **counters** via ``run_signals.incr`` (``backend\reporting\run_signals.py:22``), popped at report time. **No processing time is written to the DB or the report.**

Ad-hoc ``time.*`` timing that DOES exist (all in scripts/eval)

- ``scripts\nightly_reelcount.py:433-435`` ? ``t0 = time.monotonic(); result = ...process_video(...); wall = time.monotonic() - t0`` ? per-run ``wall_seconds`` (the real perf harness, see Q6).
- ``scripts\run_phase3_eval.py:193,206`` ? ``time.time()`` around segmentation per config sweep.
- ``scripts\run_process_and_stitch.py:70-72`` ? times the MD5 only.
- ``scripts\verify_compute_path.py:99,102`` ? ``time.monotonic()`` for a poll timeout (not perf).

Bottom line for your benchmark: nothing in the live pipeline emits stage timings you can read back; you will need to wrap the four stages yourself (or reuse ``RunTelemetry`` from ``backend\utils\run_telemetry.py``, which is already GPU-health aware and designed for exactly this).

Q4 ? Segmenters and local-GPU feasibility (8 GB RTX 2070)

Registered in ``backend\pipeline\segmenters\__init__.py``; each registers under its ``name``:

Segmenter (<code>`--segmenter`</code> value)	Class / file:line	Local?	Needs cloud key?	GPU?
<code>`motion`</code>	<code>`motion.py:14`</code> , reg <code>`:243`</code>	? fully local	No	No (pure OpenCV CPU)
<code>`gemini`</code>	<code>`gemini.py:75`</code> , reg <code>`:707`</code>	? cloud	Yes <code>`GEMINI_API_KEY`</code> (<code>`gemini.py:106-113`</code>)	No local GPU
<code>`yolo_hybrid`</code>	<code>`yolo_hybrid.py:48`</code> , reg <code>`:596`</code>	partial	Yes, always	
	(<code>`yolo_hybrid.py:87-98`</code> , no skip path)	? torch/CUDA (PersonDetector)		
<code>`wasb_hybrid`</code>	<code>`wasb_hybrid.py:25`</code> , reg <code>`:68`</code>	? if <code>`skip_ai_handoff`</code>	Only if handoff on	? (WSL2 or native)
<code>`tracknet_hybrid`</code>	<code>`tracknet_hybrid.py:23`</code> , reg <code>`:48`</code>	? if <code>`skip_ai_handoff`</code>	Only if handoff on	? (WSL2 only)
<code>`fusion_hybrid`</code>	<code>`fusion_hybrid.py:41`</code> , reg <code>`:243`</code>	? if <code>`skip_ai_handoff`</code>	Only if handoff on	? (WSL2 or native)

Key mechanics:

- **The hybrid trajectory segmenters skip the cloud entirely when ``indexing.skip_ai_handoff=True``** ? ``backend\pipeline\segmenters\trajectory_hybrid.py:199-208`` sets ``model_name="local-noai`` and does no provider call / no API key. With the default ``False``, they require ``GEMINI_API_KEY`` for the Phase-3 handoff (``trajectory_hybrid.py:209-225``).
- **``yolo_hybrid`` has no such escape** ? it unconditionally requires the key (``yolo_hybrid.py:87-98``) and returns ``Failure`` without it.
- **GPU placement:** the torchvision person detector picks CUDA at ``backend\pipeline\detectors\person_detector.py:84-85`` (``self.device = "cuda" if use_gpu and torch.cuda.is_available() else "cpu"``). WASB/TrackNet shuttle detection runs torch/CUDA either inside **WSL2** (``detector_impl="auto"``, the default ? ``tracknet_runner.py``/``wasb_runner.py``) or **Linux-native** (``detector_impl="native"`` ? ``backend\pipeline\detectors\native_wasb_runner.py``). That native runner's docstring explicitly records validation **on an RTX 2070** (``native_wasb_runner.py:30-32``) ? but via WSL; on your Windows box the practical GPU path for wasb/tracknet is ``detector_impl="auto"`` (WSL2).
- **Defaults:** ``config.json:26`` sets ``default_segmenter: "gemini"`` (note the code default in ``models.py:203`` is ``"yolo_hybrid"`` ? ``config.json`` wins). Per-run override flag: ``--segmenter`` at ``backend\main.py:1316-1320`` (choices = registered names).

Truly-local, no-key options on the RTX 2070: ``motion`` (CPU) or ``wasb_hybrid`` / ``tracknet_hybrid`` / ``fusion_hybrid`` with ``skip_ai_handoff=true`` + ``use_gpu=true`` + ``detector_impl="auto"`` (WSL2). ``gemini`` and ``yolo_hybrid`` both need the cloud key.

Q5 ? Config surface for a deterministic, all-local run

Config file: `config.json`; typed models: `backend\config\models.py`.

****Segmenter selection****

- `indexing.default\_segmenter` ? `config.json:26`; `models.py:203`. Plus CLI `--segmenter` (`main.py:1316`).

****Force fully-local / no cloud****

- `indexing.skip\_ai\_handoff` ? `models.py:289` (default `False`). Set `true` to run wasb/tracknet/fusion with zero cloud dependency.
- `deployment.compute\_target` ? `models.py:530`, `Literal["", "local\_gpu", "cloud\_run", "cpu\_mock"]` (default `""`). Leave `""` or `"local\_gpu"` to keep compute in-process.
- `deployment.phase\_placement.{validation,segment,compile}` ? `models.py:498-510`, each `control\_plane | cloud\_run\_l4`. Resolver: `backend\config\placement.py:42`
`resolve\_phase\_placement`. With `compute\_target=""` and no `phase\_placement`, the legacy defaults run ****all three phases locally / in-process**** (`placement.py:16-23`).

****Compute placement / GPU knobs****

- `indexing.use\_gpu` ? `config.json:27`; `models.py:204`.
- `indexing.detector\_impl` ? `models.py:210`, `auto | stub | native` (env `RALLY\_DETECTOR\_IMPL` overrides). `stub` = deterministic CPU mock (GPU/WSL-free); `auto` = WSL2 GPU; `native` = Linux-native GPU.
- `indexing.wasb.device` ? `models.py:91` (default `"cuda"`; env `WASB\_DEVICE`);
`indexing.wasb.weights\_path` ? `config.json:61`; `models.py:80`.

****Validation thresholds****

- `validation.min\_blur\_threshold` ? `config.json:9`; `models.py:28`.
- `validation.min\_resolution\_width/height`, `validation.min\_fps`,
`validation.max\_scene\_cuts\_per\_minute` ? `config.json:6-10`; `models.py:24-45`
(`ValidationConfig`).
- `validation.relevance.court\_green\_lower/upper`, `validation.relevance.court\_pixels\_min\_ratio`
? `config.json:11-23`; `models.py:18-21` (`ValidationRelevanceConfig`).

****Stitching****

- `stitching.begin\_padding` / `end\_padding` ? `config.json:73-74`; `models.py:376-377`.
- `stitching.min\_shot\_count` ? `config.json:76`; `models.py:379` (the floor applied in
`\_resolve\_compile`).
- `stitching.watermark.\*` (`enabled`, `text`, `position`, `opacity`, `?`) ? `config.json:77-84`;
`models.py:332`/`:408` (`WatermarkConfig`).
- `stitching.encoder` ? `models.py:395` (`libx264 | h264\_nvenc | hevc\_nvenc`, default
`libx264`); `stitching.single\_pass\_stitch` ? `models.py:407` (default `True`, ~12-14% faster
one-pass stitch).

****Determinism caveat****

- `motion`'s metadata RNG is unseeded; the nightly harness closes it with `random.seed(0)`
(`scripts\nightly\_reelcount.py:424`). Pure-CV/WASB detection is byte-deterministic for a fixed
`(video, config, weights)` on a ****fixed GPU**** (`native\_wasb\_runner.py:28-34`). `gemini` and
`yolo\_hybrid` (LLM handoff) are ****not**** deterministic ? avoid them for a repeatable benchmark;
prefer `wasb\_hybrid` with `skip\_ai\_handoff=true`, or `motion` with `random.seed(0)`.

Q6 ? Existing regression / nightly / benchmark harnesses

Harness	File	Purpose	Per-run timing?
Test suite	`run_all_tests.py`	pytest wrapper; offline/mockd (no GPU/net/WSL, `:16`)	No
Accuracy eval	`run_eval.py`	Scores stored DB predictions vs GT annotations (precision/recall/F1, IoU). No pipeline run (`:33-37`)	No
Accuracy gate	`run_regression.py`	Compares P/R/F1 to a snapshotted baseline; exit 1 on regression	No
Nightly re-score	`scripts\nightly_regression.py`	CPU re-score of **cached** WASB trajectories (windowing/scorer only) vs frozen baseline (`:1-8`). Writes `output\nightly_regression\<date>.{json,md}`	No (CPU-seconds mention only)

```
| **Nightly reel-count + cost** | `scripts\nightly_reelcount.py` | **The perf harness.** Runs
the configured segmenter E2E on golden videos; asserts reel-count vs baseline; scores quality;
measures **wall_seconds** and **USD/INR cost** | **Yes** |
| Config sweep | `scripts\run_phase3_eval.py` | Sweeps configs, times segmentation
(`time.time()`, `:193/:206`) | Yes (ad hoc) |
| Compute-path proof | `scripts\verify_compute_path.py` | Proves which compute path an
`/api/process` job took | No (poll timeout only) |
```

`scripts\nightly\_reelcount.py` is what you'll want to model your benchmark on. It already:

```
- Times each run: `backend\...\nightly_reelcount.py:433-435`
```python
t0 = time.monotonic()
result = seg_cls(db, config).process_video(video_id, local)
wall = time.monotonic() - t0
```
- Emits per-video `{name, reel_count, ok, error, quality, wall_seconds}` (`:111`, `:532-539`),
aggregates to `output\nightly_reelcount\<date>.{json,md}` (`DEFAULT_REPORT_DIR` `:53`).
- Converts wall-seconds ? cost: `cost_report(...)` `:76-100` (via `backend.billing`, USD + INR).
- Has a determinism re-run guard (`:482-495`) and seeds `random.seed(0)` (`:424`).
- Baselines/manifest: `eval_baselines\reelcount_manifest.json`,
`eval_baselines\reelcount_baseline.json` (`:51-52`).
```

****Gap for your goal:**** `nightly\_reelcount.py` measures ****segmentation only**** ? it calls
`process\_video` and counts/scores segments; it does ****not**** validate, and it does ****not****
stitch. No existing harness times all four stages (checksum/validate/segment/stitch) together.
The `output\nightly\_regression\` directory already contains dated JSON+MD reports (2026-06-18 ?
06-23) from the accuracy nightly, and `output\run\_telemetry.jsonl` is the fusion-golden
`RunTelemetry` output ? both are useful templates, but neither is a full-pipeline latency
benchmark.

Concrete recommendation for a repeatable performance-benchmark script

Deterministic config: `default\_segmenter=wasb\_hybrid` (or `motion`),
`indexing.skip\_ai\_handoff=true`, `use\_gpu=true`, `detector\_impl=auto`,
`deployment.compute\_target=""` (all-local). Then, per input video, wrap the four call sites ?
`compute\_video\_id` (`main.py:1508`), `registry.run\_all\_with\_context` (`main.py:1513`),
`segmenter.process\_video` (`main.py:1552`), and
`VideoCompiler(...).compile_highlights(video_path, [(s["start_time"], s["end_time"]) for s in
db.query_play_segments(video_id, {})], "highlights.mp4")` (Q2 Option A) ? in `time.monotonic()`
(or reuse `RunTelemetry` from `backend\utils\run\_telemetry.py:135` for GPU-health-annotated
phase snapshots). This stitches the entire reel with zero user interaction, which no current CLI
entry point does.